

UNIVERSITÀ TELEMATICA e-Campus

Corso di Laurea in

INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE (DM 1648/23)

Tesi di Laurea

**Sicurezza nei sistemi Internet of Things: architetture,
vulnerabilità e strategie di mitigazione**

Relatore: Prof. Oleksandr Kuznetsov

Candidato: Samuele Marchitelli

Matricola: 001814763

Anno Accademico 2025/2026

AUTORIZZAZIONE ALLA CONSULTAZIONE DELLA TESI DI LAUREA

Il sottoscritto Samuele Marchitelli, matricola n. 001814763,
autore della tesi dal titolo:

“Sicurezza nei sistemi Internet of Things: architetture, vulnerabilità e strategie di mitigazione”

☒ AUTORIZZA

☐ NON AUTORIZZA

la consultazione della tesi, fatto divieto di riprodurre parzialmente o integralmente il contenuto senza corretta attribuzione.

Dichiara inoltre di:

☒ AUTORIZZARE

☐ NON AUTORIZZARE

l'Università Telematica eCampus, nei limiti previsti dalla normativa vigente, al trattamento, alla comunicazione, alla diffusione e alla pubblicazione dei dati personali per le finalità istituzionali connesse alla tesi di laurea.

Data: 26/08/2026

Firma:

Abstract

La crescente diffusione dei sistemi di Internet of Things amplia la superficie di attacco delle reti, nelle quali dispositivi con risorse limitate, configurazioni deboli e aggiornamenti discontinui possono diventare punti di ingresso per attività malevole. Questa tesi analizza l'impiego di Intrusion Detection Systems basati su machine learning e sviluppa una pipeline riproducibile per la classificazione del traffico IoT sul dataset TON_IoT/UNSW, utilizzando il file `train_test_network.csv` in modalità binaria e multiclasse.

La metodologia consiste nella validazione dei dati, nel preprocessing per prevenire la contaminazione tra addestramento e test, nella codifica delle variabili categoriche, nella normalizzazione insieme a uno splitting stratificato e alla valutazione tramite accuracy, precision, recall, F1-score, Macro F1 e Weighted F1. Vengono confrontati Random Forest, LightGBM, XGBoost e Logistic Regression, che funge da baseline. Sono inoltre descritti esperimenti aggiuntivi riguardanti SMOTE, la sintonizzazione degli iperparametri, MLPClassifier e le misure di Isolation Forest, l'analisi SHAP tra le architetture di modello, le misurazioni di latenza e dimensione degli artefatti durante l'esecuzione per ciascuna delle pipeline di apprendimento automatico (in particolare includendo la compressione, grazie alla sua forte relazione con il dimostratore embedded simulato su ESP32 utilizzando Wokwi). I risultati mostrano prestazioni molto elevate per i modelli ensemble, in particolare con LightGBM che si posiziona al primo posto sia nella baseline binaria sia in quella multiclasse. Logistic Regression ottiene risultati inferiori rispetto ai modelli ensemble, ma rimane interessante per semplicità, bassa latenza e maggiore facilità di portabilità. Il confronto tra la pipeline Python e la versione quantizzata mostra il compromesso richiesto dall'implementazione embedded. I risultati vengono infine discussi in relazione a strategie di mitigazione quali segmentazione della rete, hardening, aggiornamenti del firmware, autenticazione robusta, inventario degli asset, logging e principi zero trust.

Il lavoro costituisce una base sperimentale completa e riproducibile; traffico reale, drift e validazione su hardware fisico restano sviluppi futuri.

Indice

Elenco delle figure	vii
Elenco delle tabelle	viii
Capitolo 1 - Introduzione	1
1.1 Contesto dei sistemi Internet of Things	1
1.2 Ambienti connessi e crescita ampliata della superficie di attacco	1
1.3 Problema della sicurezza IoT	2
1.4 Obiettivo e contributo della tesi	3
1.5 Struttura dell'elaborato	3
Capitolo 2 - Stato dell'arte	4
2.1 Architetture IoT	4
2.2 Protocolli e comunicazione	4
2.3 Vulnerabilità tipiche	5
2.4 Gli attacchi in ambienti IoT	5
2.5 Sistemi di Rilevamento delle Intrusioni	6
2.6 Machine learning applicato agli IDS	6
2.7 Requisiti di sicurezza in un ambiente IoT	7
2.8 IDS in architetture edge e cloud	7
2.9 Difficoltà specifiche del machine learning per la sicurezza	8
Capitolo 3 - Metodologia	9
3.1 Organizzazione del repository	9
3.2 Dataset utilizzato	9
3.3 Validazione dei dati	9
3.4 Preprocessing e prevenzione del data leakage	10
3.5 Split sperimentale	12
3.6 Modelli baseline	12
3.7 Esperimenti estesi	12

3.8 Metriche di valutazione	13
3.9 Protocollo di esecuzione	13
3.10 Gestione degli artefatti.....	14
3.11 Simulazione IoT e dimostrazione embedded	14
Capitolo 4 - Risultati sperimentali	15
4.1 Premessa sui risultati	15
4.2 Risultati baseline nella classificazione binaria	15
4.3 Risultati baseline nella classificazione multiclasse	17
4.4 Effetto del bilanciamento tramite SMOTE	20
4.5 Tuning iperparametrico.....	21
4.6 Modelli aggiuntivi: MLPClassifier e Isolation Forest	22
4.7 Interpretabilità dei modelli.....	24
4.8 Latenza, dimensione e compressione.....	26
4.9 Dimostratore embedded con Wokwi.....	27
4.10 Sintesi degli artefatti prodotti	28
4.11 Lettura critica dei risultati	28
Capitolo 5 - Discussione e strategie di mitigazione.....	30
5.1 Interpretazione complessiva dei risultati	30
5.2 Interpretabilità contro prestazioni	30
5.3 Limiti del dataset statico	31
5.4 Limiti della simulazione Wokwi.....	31
5.5 Strategie di mitigazione	31
5.6 Possibile percorso verso una validazione su hardware reale	32
5.7 Indicazioni operative per un IDS IoT	33
5.8 Considerazioni etiche e di gestione dei dati.....	33
Capitolo 6 - Conclusioni	34
Bibliografia	35

Elenco delle figure

Figura 1 - Distribuzione delle classi binarie prima del preprocessing.....	11
Figura 2 - Distribuzione delle classi binarie dopo il preprocessing.....	11
Figura 3 - Confronto dell'accuracy dei modelli baseline nella classificazione binaria...	16
Figura 4 - Confronto del Macro F1 dei modelli baseline nella classificazione binaria ..	16
Figura 5 - Matrice di confusione LightGBM nella classificazione binaria	17
Figura 6 - Confronto dell'accuracy dei modelli baseline nella classificazione multiclasse	18
Figura 7 - Confronto del Macro F1 dei modelli baseline nella classificazione multiclasse	19
Figura 8 - Matrice di confusione LightGBM nella classificazione multiclasse	20
Figura 9 - Confronto Macro F1 con e senza SMOTE.....	21
Figura 10 - Confronto Macro F1 dei modelli sottoposti a tuning binario.....	22
Figura 11 - Confronto Macro F1 degli esperimenti estesi binari.....	23
Figura 12 - Confronto Macro F1 degli esperimenti estesi multiclasse	23
Figura 13 - Importanza delle feature nel modello LightGBM binario	25
Figura 14 - Importanza delle feature nel modello LightGBM multiclasse.....	26

Elenco delle tabelle

Tabella 1 - Prestazioni dei modelli baseline nella classificazione binaria	15
Tabella 2 - Prestazioni dei modelli baseline nella classificazione multiclasse	17

Capitolo 1 - Introduzione

1.1 Contesto dei sistemi Internet of Things

Con Internet of Things, non intendiamo soltanto la raccolta di dispositivi fisici connessi a una rete, in grado di raccogliere dati dall'ambiente e di elaborarli con componenti locali o di trasmetterli ad altri sistemi. A differenza dei computer tradizionali, la maggior parte dei dispositivi IoT viene creata per un'unica finalità: misurare temperatura, umidità, pressione, vibrazioni, consumo energetico, posizione, movimento, apertura delle porte, qualità dell'aria o parametri industriali. La potenza degli asset fisici viene acquisita trasformando processi grezzi in dati digitali composti e fruibili che applicazioni, dashboard e macchine per il processo decisionale possono utilizzare.

Esistono numerosi scenari applicativi. Le smart home sono sistemi di sicurezza domestica, sistemi di illuminazione/attuazione e controllo di riscaldamento e raffreddamento tramite vari elementi di inoltro (sensori). I dispositivi distribuiti nelle smart city assistono nel monitoraggio del traffico, nell'illuminazione pubblica, nel parcheggio intelligente e nella gestione ambientale. Per quanto riguarda il settore sanitario, strumenti connessi potrebbero assemblare dati clinici e facilitare il monitoraggio a distanza. L'Internet of Things utilizzato nell'industria è di aiuto alla manutenzione predittiva, al controllo dei processi e alla raccolta dei dati delle macchine. In tutti e tre questi casi, l'affidabilità del sistema dipende non soltanto dalla correttezza funzionale, ma anche da comunicazioni e dispositivi sicuri.

L'elemento che rende gli ambienti IoT particolarmente complessi è l'eterogeneità. Una stessa infrastruttura può includere microcontrollori, dispositivi embedded Linux, gateway industriali, reti wireless, broker MQTT, applicazioni cloud, API, database e interfacce web. Ogni componente utilizza tecnologie, protocolli e cicli di aggiornamento differenti. Di conseguenza, la sicurezza IoT non può essere affrontata come un problema isolato del singolo dispositivo: deve essere analizzata a livello architetturale, considerando l'intero percorso del dato, dalla raccolta alla trasmissione, dall'elaborazione alla conservazione [15].

1.2 Ambienti connessi e crescita ampliata della superficie di attacco

Gli ambienti connessi hanno ampliato la superficie di attacco, creando più punti di vulnerabilità. Ogni dispositivo sulla rete è un possibile punto di ingresso, in particolare

quando utilizza credenziali deboli, firmware non aggiornato, servizi di rete esposti o configurazioni predefinite. La situazione è ancora peggiore perché molti dispositivi IoT vengono installati e lasciati senza supervisione e funzionano per anni senza aggiornamenti, senza alcun monitoraggio, né un sistema centralizzato di gestione delle risorse.

La rete tradizionale presenta strumenti consolidati, strumenti di sicurezza per il logging che gestiscono workstation e server con gestione delle patch e controllo degli accessi. Tuttavia, in una rete IoT i dispositivi possono essere meno visibili agli amministratori, più difficili da aggiornare e non dispongono di agenti di sicurezza installabili localmente. Inoltre, alcune reti IoT includono un'ampia gamma di dispositivi economici con memoria e CPU limitate, in cui algoritmi crittografici diversi e costosi o sistemi di monitoraggio avanzati sarebbero molto lontani dall'essere disponibili.

Di conseguenza, la superficie di attacco cresce non solo per il numero di dispositivi coinvolti, ma anche per la varietà delle possibili vulnerabilità. Un dispositivo IoT compromesso può consentire a un attaccante di muoversi lateralmente nella rete, partecipare a una botnet, generare traffico dannoso o alterare dati provenienti dal mondo fisico. In ambito industriale ciò può compromettere la continuità operativa; in ambito sanitario può incidere sulla riservatezza dei dati e sulla sicurezza del paziente; nelle infrastrutture urbane può influire sulla disponibilità dei servizi pubblici.

1.3 Problema della sicurezza IoT

La sicurezza IoT presenta difficoltà specifiche. In primo luogo, molti dispositivi sono progettati privilegiando costo, autonomia energetica e facilità di installazione. Questi obiettivi sono legittimi dal punto di vista commerciale, ma possono portare a trascurare aspetti come autenticazione forte, aggiornamenti sicuri, cifratura del traffico, logging e protezione delle credenziali. In secondo luogo, la lunga vita operativa di alcuni dispositivi rende difficile mantenere un livello di sicurezza coerente nel tempo.

Un ulteriore problema riguarda la visibilità. Per proteggere una rete è necessario sapere quali dispositivi sono presenti, quali servizi espongono, quali protocolli utilizzano e quale comportamento di traffico è normale. Negli ambienti IoT questa informazione non è sempre disponibile. Dispositivi installati in momenti diversi, da fornitori diversi e con finalità diverse possono convivere nella stessa rete senza una governance uniforme.

La sicurezza deve quindi combinare misure preventive e misure di rilevamento. Le prime includono hardening, configurazioni sicure, segmentazione, autenticazione, aggiornamenti e controllo accessi. Le seconde includono monitoraggio, logging, sistemi IDS e analisi del traffico. Gli Intrusion Detection Systems sono particolarmente importanti perché consentono di individuare attività sospette anche quando le misure preventive non sono sufficienti a impedire la compromissione [3], [17].

1.4 Obiettivo e contributo della tesi

Lo scopo della tesi è creare, applicare e documentare una pipeline sperimentale per l'uso di modelli di machine learning nell'intrusion detection in ambienti IoT. Il lavoro parte dall'analisi del repository di riferimento indicato dal relatore e dalla riproduzione della relativa pipeline principale. Il codice e la documentazione sono stati riorganizzati per adattarli al contesto di una tesi sperimentale. Il contributo è articolato in tre livelli. Il primo riguarda la riproduzione della baseline: elaborazione del dataset TON_IoT, classificazione binaria e multiclasse, addestramento dei modelli principali e generazione delle metriche. Il secondo amplia il confronto tramite SMOTE, ottimizzazione degli iperparametri, MLPClassifier e Isolation Forest. Il terzo riguarda interpretabilità e portabilità, includendo importanza delle feature, misure di latenza, quantizzazione del modello e un dimostratore embedded simulato in Wokwi.

L'obiettivo non è realizzare un IDS pronto per il deployment in una rete reale, ma costruire un framework sperimentale riproducibile e verificabile dal punto di vista tecnico. Tutte le metriche riportate sono ottenute eseguendo il codice e vengono registrate nei file CSV generati dagli script.

1.5 Struttura dell'elaborato

Il Capitolo 2 presenta lo stato dell'arte relativo ad architetture IoT, vulnerabilità, attacchi e sistemi IDS. Il Capitolo 3 descrive la metodologia sperimentale, il dataset, il preprocessing, i modelli e il protocollo di valutazione. Il Capitolo 4 riporta i risultati ottenuti, includendo tabelle e figure generate dalla pipeline. Il Capitolo 5 discute i risultati, i limiti sperimentali e le strategie di mitigazione. Il Capitolo 6 conclude il lavoro e individua possibili sviluppi futuri.

Capitolo 2 - Stato dell'arte

2.1 Architetture IoT

Le architetture IoT sono generalmente visualizzate nel formato a lettura a strati. Lo strato di percezione contiene sensori e attuatori che interagiscono direttamente con il mondo fisico. È lo strato di protocolli e tecnologie di rete che consente ai dispositivi di inviare informazioni. Lo strato edge (o gateway) aggrega, filtra e svolge informazioni preliminari. Ultimo ma non meno importante, lo strato applicativo che consiste in piattaforme cloud, dashboard, sistemi di analisi e applicazioni per l'utente finale [15].

Questo tipo di scomposizione è utile, perché mostra come la sicurezza debba essere applicata lungo l'intera catena. In sintesi, un sensore può essere manomesso fisicamente, il canale wireless può essere intercettato facilmente, granulare“granulare”, non protetto può essere connesso“non” protetto può essere connesso e le API cloud possono esporre informazioni sensibili.

Il gateway è un elemento centrale nel contesto dell'IoT. Quest'ultimo non può solo effettuare traduzione dei protocolli, ma anche filtraggio, buffering, aggregazione e inoltrare verso il cloud. Nel caso di IDS, distribuire la funzione il più in alto possibile nell'architettura di rete ha un impatto positivo sulle prestazioni perché i gateway sono in grado di gestire molte più computazioni rispetto ai computer dei nodi sensore e possono vedere numerosi flussi di traffico.

2.2 Protocolli e comunicazione

I sistemi IoT utilizzano protocolli diversi in base ai requisiti di consumo energetico, portata, latenza e affidabilità. Wi-Fi ed Ethernet sono comuni negli ambienti domestici e industriali; Bluetooth Low Energy è adatto alle comunicazioni a corto raggio; Zigbee e Thread sono diffusi nelle reti mesh; LoRaWAN consente comunicazioni a lunga distanza con basso consumo energetico; MQTT è ampiamente utilizzato come protocollo applicativo basato sul modello publish/subscribe.

MQTT è particolarmente rilevante perché consente a dispositivi leggeri di pubblicare messaggi su topic gestiti da un broker. Il modello publish/subscribe semplifica la comunicazione, ma introduce rischi se il broker è esposto senza autenticazione, senza TLS o con autorizzazioni troppo permissive. Un attaccante potrebbe pubblicare messaggi falsi, iscriversi a topic sensibili o generare traffico anomalo. Per questo

motivo, la sicurezza del broker e del server è un elemento importante nelle architetture IoT [4].

2.3 Vulnerabilità tipiche

Le credenziali deboli o predefinite rappresentano una delle cause più comuni di vulnerabilità nei sistemi IoT, insieme a servizi non necessari abilitati, firmware obsoleto, assenza di cifratura, gestione inadeguata delle chiavi, API non protette e configurazioni cloud insufficienti. Queste vulnerabilità non agiscono in modo isolato, ma possono combinarsi tra loro e creare vettori di attacco [3], [15], [17]. Un caso frequente riguarda un dispositivo esposto in rete con credenziali predefinite. Dopo aver ottenuto l'accesso, un attaccante può modificare la configurazione, installare malware oppure utilizzare il dispositivo per generare traffico verso altri sistemi. Se la rete non è segmentata, il dispositivo compromesso può diventare un punto di accesso ai sistemi principali. In assenza di registrazione centralizzata degli eventi, l'attacco può rimanere inosservato. Il firmware obsoleto costituisce un ulteriore rischio: molti dispositivi restano senza patch anche dopo la pubblicazione di correzioni da parte del produttore, a causa dell'assenza di procedure automatiche, del timore di interruzioni del servizio o della mancanza di manutenzione. In ambito industriale, esigenze di continuità operativa possono inoltre ritardare l'applicazione delle patch.

2.4 Gli attacchi in ambienti IoT

Gli attacchi sono diversi negli ambienti IoT. Per esempio, quelli di scansione sono progettati per individuare servizi e dispositivi esposti. Gli attacchi di brute-force, invece, sfruttano credenziali deboli. Gli attacchi di Denial of Service (DoS) o di Distributed Denial of Service (DDoS) sono focalizzati sulla compromissione della disponibilità. Gli attacchi di spoofing e man-in-the-middle modificano o intercettano le comunicazioni. Le botnet IoT utilizzano dispositivi compromessi per generare traffico coordinato. Questi attacchi possono riflettersi in pattern di traffico osservabili, come un aumento del numero di connessioni, la modifica della porta di destinazione, volumi anomali di byte, sequenze anomale di pacchetti, durata anomala del flusso o cambiamenti nella distribuzione della classe del traffico dal punto di vista di un IDS. Ecco perché TON_IoT e altri dataset di rete possono essere utili per addestrare e poi valutare modelli di classificazione.

2.5 Sistemi di Rilevamento delle Intrusioni

Un Intrusion Detection System (IDS) è responsabile del rilevamento di attività sospette o dannose. Esistono gli IDS basati sull'host che analizzano eventi che avvengono su un singolo dispositivo e poi gli IDS basati sulla rete che monitorano il traffico che scorre sulla rete. In generale, c'è una scelta progettuale su dove collocare l'IDS nel contesto IoT: può essere installato direttamente sul dispositivo (basso costo di latenza ma maggiore consumo di risorse), su gateway o nodi edge estremamente capaci per tale scopo che eseguiranno analisi distribuite, oppure affidarsi completamente all'analisi in cloud con penalità di connettività [16]. Gli IDS basati su firme ispezionano il traffico e lo confrontano con regole note. Possono intercettare attacchi che già conoscono, ma non gestiscono bene le nuove varianti. Gli IDS basati su anomalie costruiscono un modello di comportamento normale e attivano l'allarme per le deviazioni. Questi ultimi si suddividono in due categorie e il motivo per cui la seconda categoria è più adatta agli ambienti IoT è che la maggior parte dei dispositivi mostra comportamenti ripetitivi: un sensore invia misurazioni periodiche, un attuatore riceve comandi specifici e un gateway comunica con endpoint noti. Ma il "comportamento normale" cambia forma ed è difficile stabilire cosa sia normale, anche in presenza di aggiornamenti, cambiamenti operativi e rumore nei dati.

2.6 Machine learning applicato agli IDS

L'apprendimento supervisionato consente di apprendere le relazioni tra le caratteristiche del traffico e le classi corrispondenti utilizzando il machine learning. I modelli supervisionati includono un dataset con etichette che indicano se il campione è benigno o dannoso, oppure a quale tipo di attacco appartiene. Per i modelli non supervisionati, non è necessario disporre di etichette di attacco durante l'addestramento dell'algoritmo, che cerca anomalie. Random Forest, XGBoost e LightGBM sono modelli di ensemble basati su alberi decisionali [5], [6], [7]. Sono adatti ai dati tabellari e in grado di modellare relazioni non lineari. La regressione logistica è un modello lineare più semplice e interpretabile, spesso usato come baseline, e costituisce un riferimento utile rispetto a tecniche più complesse negli scenari embedded [20]. MLPClassifier: è una rete neurale feed-forward implementata in scikit-learn [21]; offre molta più flessibilità, ma richiede attenzione con la convergenza e la sintonizzazione. L'Isolation Forest è un metodo di rilevamento di anomalie non supervisionato che si basa sull'ipotesi che le anomalie siano più facili da isolare rispetto ai campioni normali [9].

2.7 Requisiti di sicurezza in un ambiente IoT

Un ambiente IoT deve essere analizzato per la sicurezza rispetto ai requisiti classici di riservatezza, integrità e disponibilità, che tuttavia presentano una natura specifica quando si tratta di dispositivi distribuiti e connessi.

La riservatezza riguarda i dati raccolti dai sensori e inviati o trasferiti verso gateway o strutture on-premises/cloud. Tali dati possono essere sensibili in diversi scenari: informazioni sanitarie, posizione, abitudini domestiche o misurazioni di processi industriali e di produzione.

L'integrità è altrettanto importante; dati modificati possono portare a decisioni errate. Misurazioni falsificate possono compromettere l'automazione locale in una smart home; dati alterati possono indebolire la manutenzione predittiva o il controllo del processo in un contesto industriale.

Ultimo ma non meno importante è la disponibilità, poiché un certo numero di dispositivi IoT funziona tramite servizi continuativi. Un attacco di Denial of Service (DoS) contro un broker, un gateway o un dispositivo può interferire con la raccolta dei dati o l'attivazione dei comandi.

Oltre alla classica triade riservatezza-integrità-disponibilità, devono essere considerate anche autenticazione, autorizzazione, tracciabilità e resilienza. L'autenticazione limita l'accessibilità del sistema solo a dispositivi e utenti autorizzati. L'autorizzazione limita ciò che puoi fare. La tracciabilità consente di ricostruire eventi e incidenti utilizzando i log. La resilienza descrive la capacità di un sistema di resistere, ma può anche continuare a funzionare in qualche modo nonostante guasti o attacchi.

2.8 IDS in architetture edge e cloud

La posizione di un IDS ha un impatto elevato su prestazioni, visibilità e costi. Un IDS installato su un dispositivo è in grado di rilevare eventi locali con bassa latenza, ma è limitato da memoria, CPU e consumo energetico. Un IDS distribuito su un gateway o su un nodo edge dispone di maggiore potenza di elaborazione e può monitorare il traffico proveniente da più dispositivi. Un IDS posizionato nel cloud può eseguire esami più avanzati; tuttavia, dipende dalla trasmissione delle informazioni e può causare inattività o problemi di sicurezza.

Molte volte, una soluzione ibrida è utile in un caso del genere. Il dispositivo può eseguire controlli semplici, il gateway può applicare modelli più complessi e il cloud può effettuare analisi storiche, correlazione degli eventi e aggiornamenti regolari dei modelli. Il framework consente di distribuire il carico di elaborazione e di personalizzare la risposta in base alla criticità delle operazioni.

Qui la tesi adotta questa impostazione: la pipeline Python rappresenta l'intero livello sperimentale, adatto sia per l'addestramento sia per la valutazione; al contrario, il dimostratore Wokwi rappresenta un livello semplificato, semplificato dell'inferenza embedded. Ciò permette di parlare del compromesso tra accuratezza e portabilità quando si confrontano questi due livelli tra loro.

2.9 Difficoltà specifiche del machine learning per la sicurezza

L'applicazione del machine learning alla sicurezza presenta difficoltà che non emergono sempre in altri ambiti. La prima riguarda lo sbilanciamento delle classi: alcuni attacchi possono essere rari, mentre il traffico normale può essere molto più frequente. In questi casi, metriche come l'accuracy possono essere insufficienti. Per questo motivo la tesi considera Macro F1 e Weighted F1, oltre alle metriche più tradizionali.

La seconda difficoltà riguarda l'evoluzione degli attacchi. Un modello può imparare pattern presenti nel dataset, ma un attaccante può modificare comportamento, porte, frequenza dei pacchetti o payload per aggirare il rilevamento. La terza difficoltà riguarda la spiegabilità: un IDS che segnala un attacco deve fornire elementi utili per l'analisi, altrimenti rischia di produrre alert difficili da gestire.

La quarta difficoltà riguarda il deployment. Un modello addestrato in Python può funzionare bene su un computer, ma essere inadatto a un microcontrollore. Memoria, latenza, consumo energetico e aggiornabilità sono vincoli pratici che devono essere considerati fin dalla progettazione. Per questo motivo, nella tesi sono state incluse anche misure di dimensione e un esempio di modello embedded.

Capitolo 3 - Metodologia

3.1 Organizzazione del repository

Il repository organizza configurazioni, dati, codice sorgente, script, documentazione dei risultati, tesi e dimostratore Wokwi. Questa separazione evita la confusione tra dati grezzi, artefatti generati e codice. La cartella `config/` contiene il file YAML con percorsi e parametri sperimentali; `data/raw/` ospita il dataset scaricato manualmente, mentre `data/processed/` può contenere dati trasformati. La directory `src/` include i moduli Python per caricamento dei dati, preprocessing, addestramento, valutazione, visualizzazione, esperimenti e simulazione. La directory `results/` raccoglie metriche, grafici, modelli e log. La documentazione accademica in italiano è contenuta in `thesis/`.

Questa configurazione evita l'uso di percorsi assoluti e consente di eseguire gli script dalla directory principale del repository. I file di grandi dimensioni e gli artefatti non adatti al versionamento sono esclusi tramite `.gitignore`.

3.2 Dataset utilizzato

Il dataset utilizzato è TON_IoT/UNSW, nella componente di rete [1], [2], [13], [14]. Il file atteso dalla pipeline è `data/raw/train_test_network.csv`. La colonna `label` viene utilizzata come target per la classificazione binaria, mentre la colonna `type` viene utilizzata come target per la classificazione multiclasse.

La classificazione binaria è progettata per separare traffico normale da traffico di attacco. D'altra parte, la classificazione multiclass è in grado di differenziare tra più categorie di traffico come classi normali e diversi tipi di attacchi. Il secondo compito è più difficile perché non deve solo rilevare l'esistenza di una minaccia, ma anche che tipo di minaccia sia.

Questo dataset non esiste nel repository, è stata presa questa decisione sia per evitare il versioning di file pesanti sia per mantenere il progetto aggiornato con le migliori pratiche di gestione dei dati. Gli script controllano se il file è presente e, se il dataset non esiste, l'esecuzione si interrompe con un messaggio chiaro senza produrre artefatti sperimentali.

3.3 Validazione dei dati

La pipeline verifica che il file atteso sia presente e che le colonne target richieste siano disponibili prima del preprocessing. In caso contrario, se si verifica un errore, indica

quale percorso del dataset è stato utilizzato e come procedere. Questo è un comportamento positivo perché evita messaggi poco chiari come le eccezioni pandas o un `KeyError`, che non hanno molto valore per chi intende eseguire il progetto per la prima volta.

La validazione dei target è importante perché i Dataset scaricati da fonti diverse potrebbero avere nomi o colonne differenti. Questo è implicito nella configurazione del progetto che mostra un'etichetta per la classificazione binaria e `type` per la classificazione multiclass. Nel caso in cui l'esecuzione della pipeline si fermasse, viene indicata la mancanza di queste colonne.

3.4 Preprocessing e prevenzione del data leakage

Se necessario, vengono applicate fasi di preprocessing (gestione dei valori mancanti, eliminazione o trattamento delle colonne non adatte all'addestramento, codifica delle variabili categoriche, scaling delle caratteristiche continue). La pipeline previene i data leakage, ossia l'uso non intenzionale di informazioni del set di test durante l'addestramento. Più precisamente, gli encoder e gli scalatori vengono adattati (fit) solo sul set di training. Successivamente, le trasformazioni apprese vengono applicate al set di test. Questo è in linea con la tipica aspettativa in un contesto reale in cui, utilizzando il modello su dati non visti, non è possibile conoscere a priori la distribuzione del set di test. SMOTE viene usato esclusivamente sui dati del set di training. Non è stato applicato SMOTE al set di test, perché ciò significherebbe alterare i dati sui quali il modello deve essere valutato e quindi produrre una valutazione non valida. Per questo motivo la pipeline mantiene separati addestramento e valutazione.

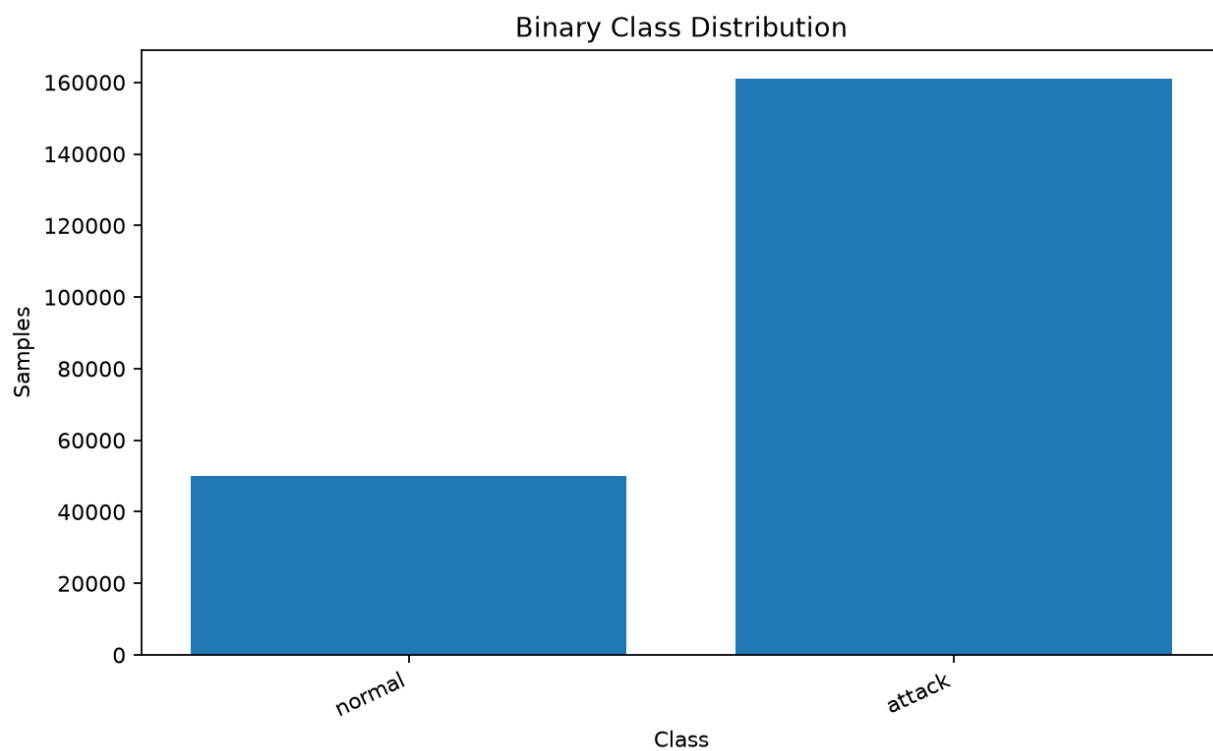


Figura 1 - Distribuzione delle classi binarie prima del preprocessing

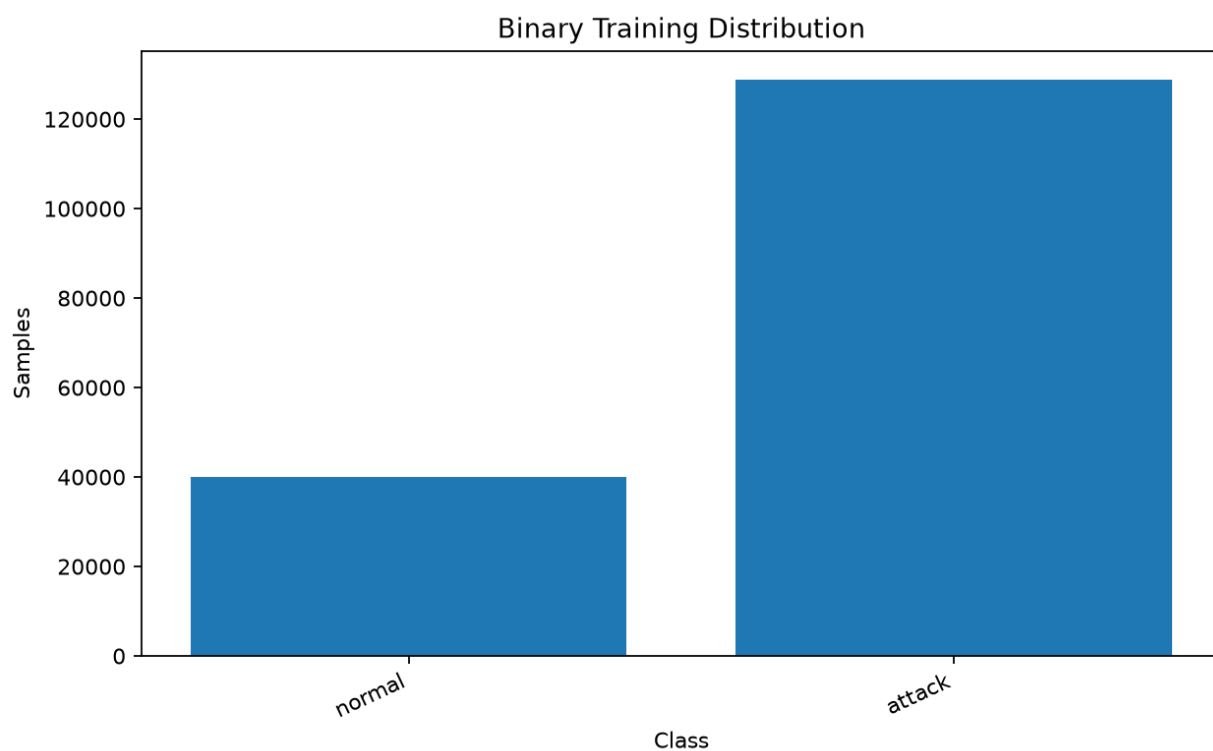


Figura 2 - Distribuzione delle classi binarie dopo il preprocessing

3.5 Split sperimentale

La suddivisione addestramento e test è stratificata, con una quota di test pari al 20% e seed casuale fissato a 42. La stratificazione mantiene una distribuzione delle classi simile tra training set e test set, riducendo il rischio che una classe minoritaria sia rappresentata in modo insufficiente in una delle due partizioni.

Il seed fisso permette di ripetere l'esperimento ottenendo la stessa suddivisione e quindi metriche confrontabili. Questa scelta è essenziale in un progetto di tesi sperimentale, perché consente al relatore o ad altri studenti di verificare i risultati.

3.6 Modelli baseline

Questi modelli di base sono: Random Forest, LightGBM, XGBoost e Logistic Regression. Random Forest costruisce un ensemble di alberi decisionali addestrati su diversi subset dei dati e ne media le previsioni. Sia LightGBM che XGBoost sono modelli avanzati di gradient boosting che imparano dai precedenti alberi in modo sequenziale. Logistic Regression: È un modello lineare che predice la probabilità di una classe. Questi sono i modelli scelti per il problema così da poterne confrontare l'uno con l'altro. Sebbene i modelli ensemble eccellano in generale sui dati tabellari, la Logistic Regression è più interpretabile e leggera. In questo modo È possibile valutare anche la semplicità, il tempo di inferenza e la possibile portabilità, oltre all'accuratezza.

3.7 Esperimenti estesi

Gli esperimenti estesi includono SMOTE, tuning iperparametrico, MLPClassifier e Isolation Forest. SMOTE affronta lo sbilanciamento generando campioni sintetici della classe minoritaria nel training set [8]. Il tuning iperparametrico esplora configurazioni alternative dei modelli ensemble per verificare eventuali margini di miglioramento. MLPClassifier introduce un modello neurale feed-forward. Isolation Forest valuta un approccio di anomaly detection non supervisionato [9].

Il dataset è composto da 211.045 record: le linee di base dello split stratificato 80/20 utilizzano 168.836 record di training e 42.209 record di test. Pertanto, SMOTE, la fase di tuning e MLPClassifier applicano campioni stratificati di 30.000 record di training sia per i casi binari sia per quelli multiclass per mantenere esperimenti estesi sostenibili e confrontabili; la valutazione viene eseguita in modo coerente sull'intero set di test indipendente di 42.209 record. Nel confronto con SMOTE, anche il training set bilanciato deve essere ricampionato a 30k record dopo il sovracampionamento. Al

contrario, Isolation Forest viene addestrato esclusivamente sui 7.108 record normali nel campione binario, poiché si comporta come un addestramento monoclasse. Questo viene valutato con una cross-validation a 3 fold su 6 configurazioni casuali. Le dimensioni reali di ciascun campione appaiono nelle colonne `training_samples` e `test_samples` nei file CSV.

Sono state inoltre introdotte analisi complementari: interpretabilità SHAP [10], misure di latenza e dimensione degli artefatti, compressione dei modelli e generazione di un modello compatto esportabile in C++ per simulazione Wokwi.

3.8 Metriche di valutazione

Le metriche principali sono accuracy, precision, recall, F1-score, Macro F1 e Weighted F1. L'Accuracy è il rapporto che indica quante previsioni sono corrette, ma può essere fuorviante su classi sbilanciate. È possibile valutare i falsi positivi e i falsi negativi usando precision e recall. Il F1-score è una metrica per bilanciare precision e recall. Il Macro F1 calcola la media non pesata tra le classi ed è utile per evidenziare i problemi con quelle minoritarie.

Weighted F1 pesa le classi in base alla loro numerosità.

Per ogni modello vengono inoltre salvate matrice di confusione, report di classificazione, tempo di training e tempo di inferenza. Le matrici di confusione consentono di analizzare quali classi vengono confuse tra loro. I tempi di training e inferenza sono importanti perché un IDS deve poter operare entro vincoli pratici, soprattutto in scenari edge o gateway.

3.9 Protocollo di esecuzione

Il protocollo sperimentale è stato definito per essere ripetibile. Dopo la preparazione dell'ambiente Python e l'installazione delle dipendenze, il dataset deve essere collocato nel percorso previsto. Successivamente, gli esperimenti possono essere eseguiti tramite gli script presenti in `src/experiments/`. Lo script `run_binary.py` esegue la baseline binaria, `run_multiclass.py` la baseline multiclasse e `run_extended.py` gli esperimenti con SMOTE, tuning, MLPClassifier e Isolation Forest.

Il nome `run_all.py` indica l'orchestrazione del nucleo classificativo: lo script richiama, nell'ordine, `run_binary.py`, `run_multiclass.py`, `run_extended.py` e `generate_result_summary.py`. Non richiama `run_interpretability.py`,

`run_deployment_analysis.py`, `export_embedded_model.py` né la simulazione MQTT/Wokwi. Queste analisi sono eseguite separatamente perché presentano dipendenze, costi computazionali o ambienti di esecuzione differenti; i comandi dedicati sono documentati nel README.

Ogni runner segue lo stesso schema logico: caricamento della configurazione, validazione del dataset, preprocessing, addestramento, valutazione, salvataggio delle metriche e generazione dei grafici. Questo schema riduce la duplicazione e consente di individuare rapidamente eventuali errori.

La pipeline è stata controllata anche rispetto al caso in cui il dataset non sia presente. In tale scenario, gli script terminano con un messaggio esplicito e non producono CSV o PNG di risultato. Questa scelta è importante dal punto di vista metodologico: in una tesi sperimentale, i risultati devono derivare da esecuzioni reali.

3.10 Gestione degli artefatti

I risultati si trovano nella cartella `results/`. Tutte le metriche tabellari invece sono salvate in `results/metrics/`, i grafici in `results/plots/`, gli artefatti del modello in `results/models/` e i log in `results/logs/`. Il repository è progettato per ospitare metriche e grafici versionati provenienti da esperimenti reali; non copre dataset grezzi, modelli addestrati o log. Questa distinzione è utile poiché separa metriche e grafici, che costituiscono la documentazione di una sessione di sperimentazione, dai dataset e dai modelli che potrebbero essere grandi, sensibili o destinati a essere riproducibili.

3.11 Simulazione IoT e dimostrazione embedded

Il progetto prevede anche una simulazione IoT con una pipeline su dati reali. La simulazione Docker/MQTT [11] dimostra che i dispositivi simulati possono pubblicare messaggi, un componente di raccolta può raccogliere tali messaggi e uno script genera sequenze di messaggi anomali senza riprodurre un attacco DDoS di rete reale. Questo componente non è pensato per sostituire TON_IoT, ma può servire come materiale aggiuntivo per la discussione architetturale. Una versione più avanzata, il dimostratore Wokwi [12], esporta un piccolo modello in C++ e lo simula su un ESP32. L'obiettivo non è ottenere prestazioni simili a LightGBM; ma piuttosto dimostrare che sia possibile incorporare un classificatore leggero in un ambiente embedded in un contesto TinyML [19]

Capitolo 4 - Risultati sperimentali

4.1 Premessa sui risultati

Tutti i valori riportati in questo capitolo derivano dai file CSV generati dalla pipeline Python dopo l'esecuzione degli esperimenti sul dataset locale. I grafici inclusi nel documento sono presenti nella directory results/plots/ e sono stati prodotti dagli script del repository.

I risultati vanno interpretati tenendo conto del contesto sperimentale. Il dataset è statico e già etichettato; le prestazioni ottenute indicano la capacità dei modelli di classificare quel dataset secondo la suddivisione adottata, non garantiscono automaticamente prestazioni identiche in reti reali con traffico variabile, dispositivi diversi o attacchi non osservati in addestramento.

4.2 Risultati baseline nella classificazione binaria

Tabella 1 - Prestazioni dei modelli baseline nella classificazione binaria

Modello	Accuracy	F1-score	Macro F1	Training (s)	Inference (s)
Random Forest	0.9988	0.9992	0.9983	24.43	0.283
LightGBM	0.9992	0.9995	0.9989	3.50	0.215
XGBoost	0.9988	0.9992	0.9983	5.46	0.066
Logistic Regression	0.9576	0.9722	0.9414	5.72	0.008

Nella classificazione binaria, LightGBM ottiene la migliore accuracy e il migliore F1-score tra i modelli baseline. Random Forest e XGBoost mostrano risultati molto vicini, con differenze minime sul piano predittivo. Logistic Regression ottiene valori inferiori, ma conserva un tempo di inferenza particolarmente ridotto.

La matrice di confusione di LightGBM mostra 9987 campioni normali classificati correttamente, 13 falsi positivi, 32188 attacchi classificati correttamente e 21 falsi negativi. Questo risultato indica un numero molto basso di errori rispetto alla dimensione del test set. In un contesto IDS, i falsi negativi sono particolarmente critici perché rappresentano attacchi non rilevati; i falsi positivi, invece, possono generare alert inutili e affaticamento operativo.

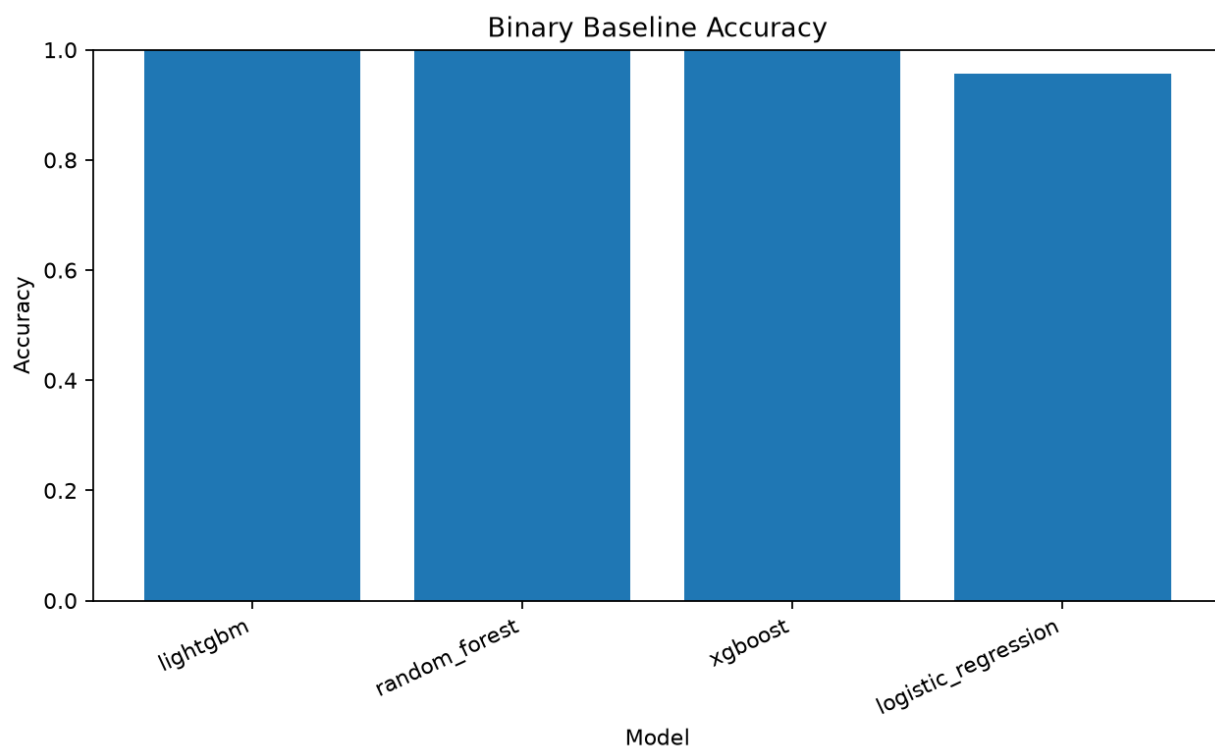


Figura 3 - Confronto dell'accuracy dei modelli baseline nella classificazione binaria

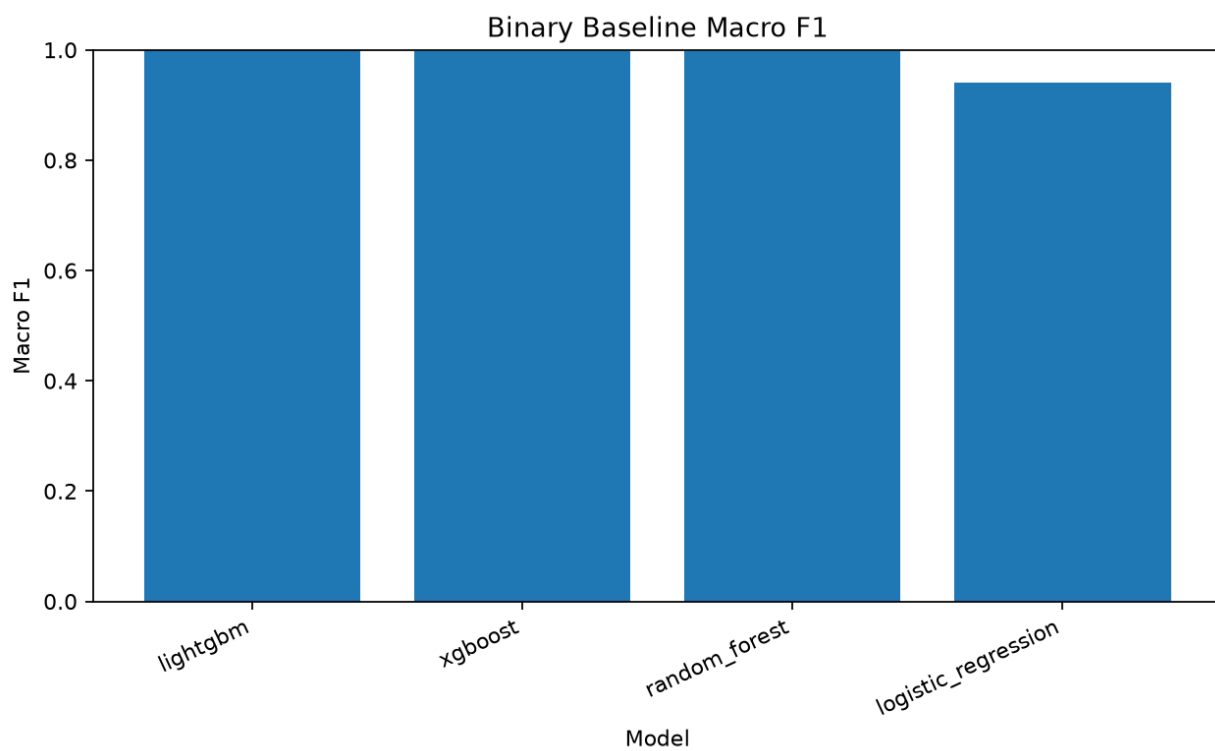


Figura 4 - Confronto del Macro F1 dei modelli baseline nella classificazione binaria

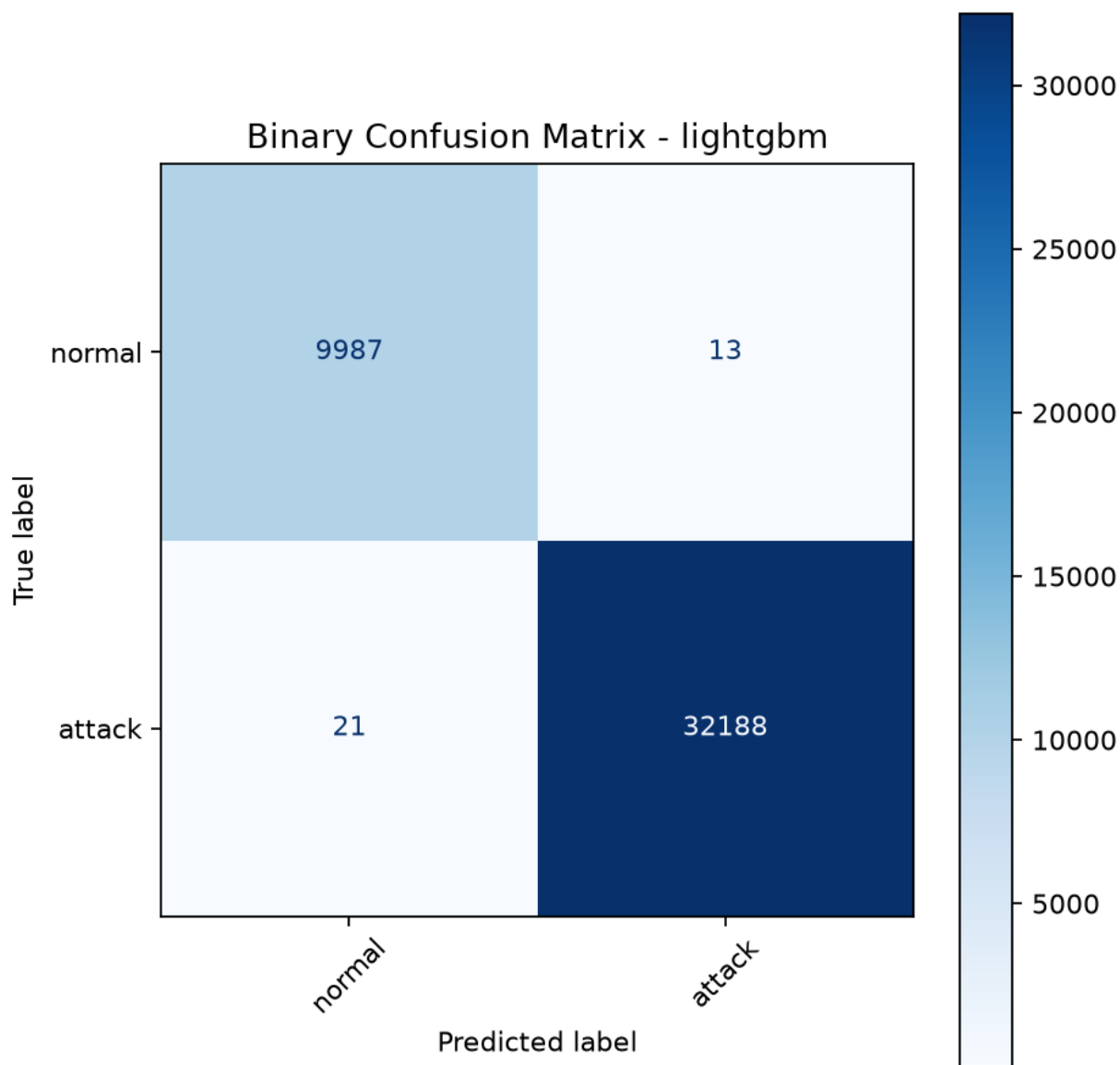


Figura 5 - Matrice di confusione LightGBM nella classificazione binaria

4.3 Risultati baseline nella classificazione multiclasse

Tabella 2 - Prestazioni dei modelli baseline nella classificazione multiclasse

Modello	Accuracy	Weighted F1	Macro F1	Training (s)	Inference (s)
Random Forest	0.9883	0.9884	0.9654	23.63	0.539
LightGBM	0.9897	0.9898	0.9678	26.82	2.422
XGBoost	0.9891	0.9893	0.9655	58.96	0.557
Logistic Regression	0.8263	0.8330	0.7686	211.00	0.022

Nella situazione multiclasse, i nostri primi risultati mostrano che LightGBM ottiene la migliore accuratezza di base e il valore Macro F1. XGBoost e Random Forest mantengono risultati competitivi, mentre Logistic Regression mostra limiti più evidenti. Tale differenza è coerente con la maggiore complessità della classificazione multiclasse: distinguere tra diverse tipologie di attacco richiede confini decisionali più articolati rispetto alla sola separazione tra traffico normale e traffico malevolo. In questo contesto

il Macro F1 è particolarmente utile, perché attribuisce lo stesso peso a tutte le classi. Un modello può fornire risultati altamente accurati classificando bene le classi più prevalenti, pur faticando con le classi minoritarie. Il fatto che LightGBM raggiunga anche il Macro F1 più alto indica una capacità più equilibrata di discriminare tra le diverse classi.

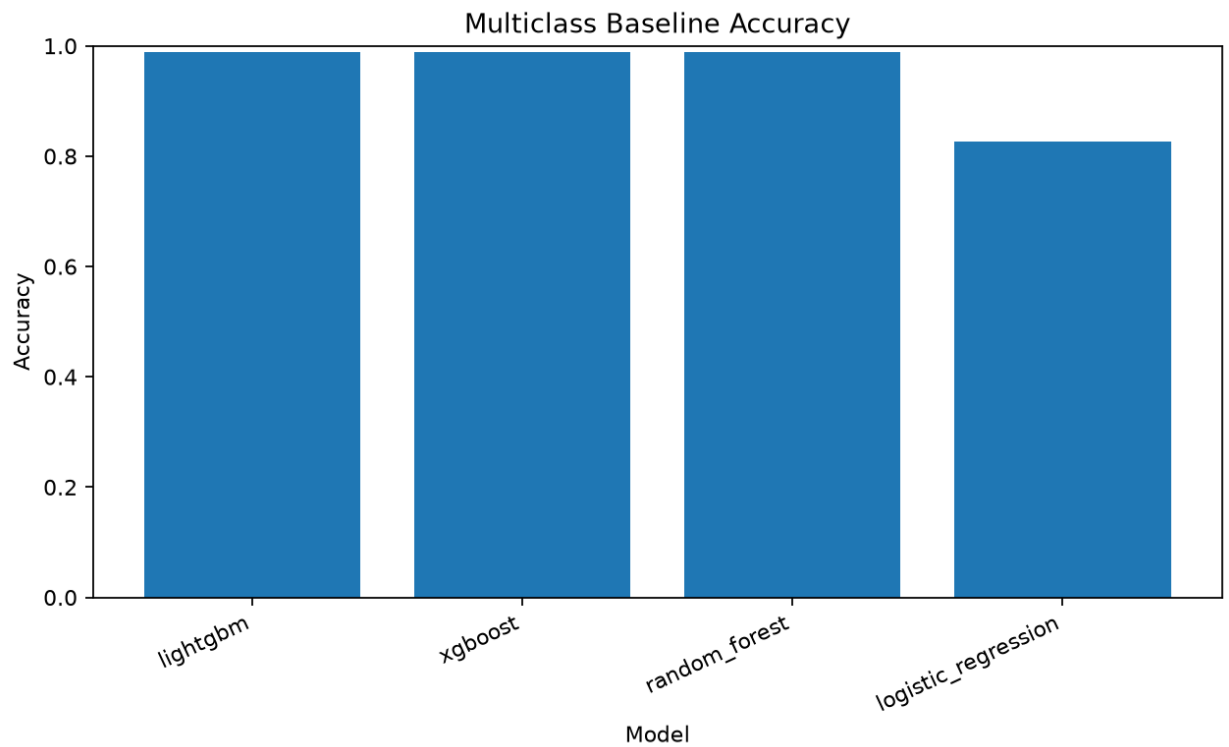


Figura 6 - Confronto dell'accuracy dei modelli baseline nella classificazione multiclasse

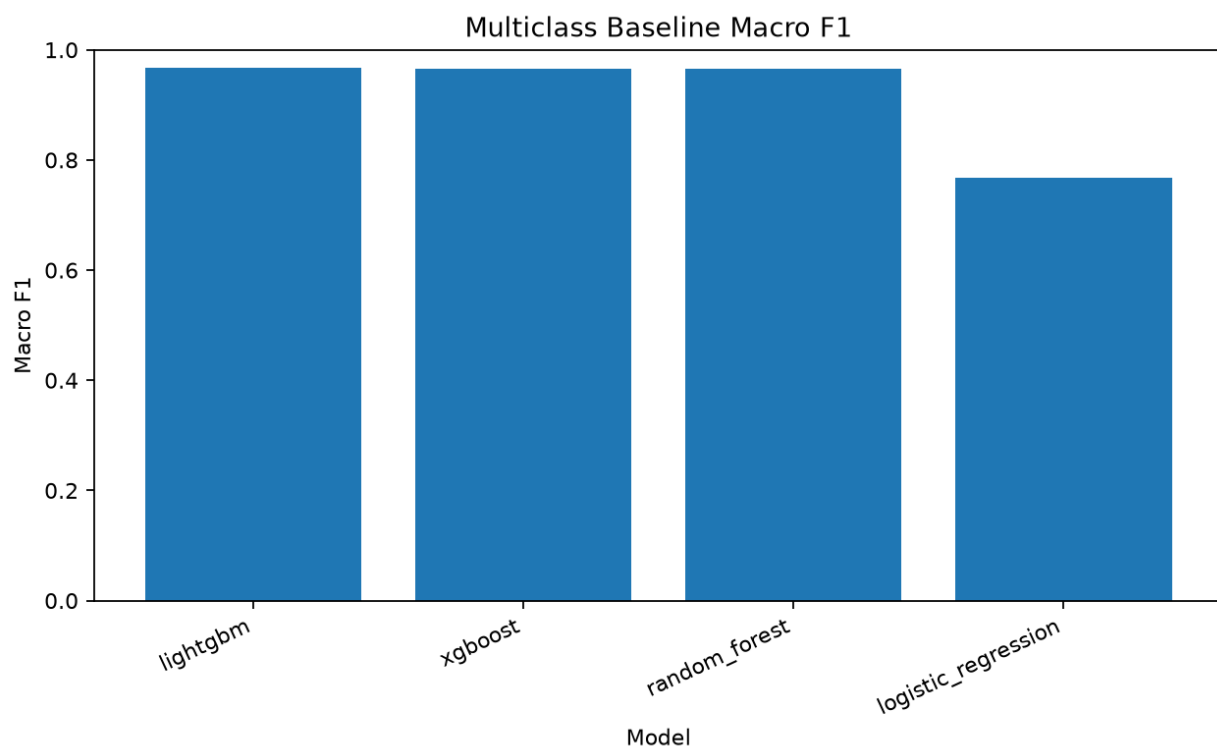


Figura 7 - Confronto del Macro F1 dei modelli baseline nella classificazione multiclasse

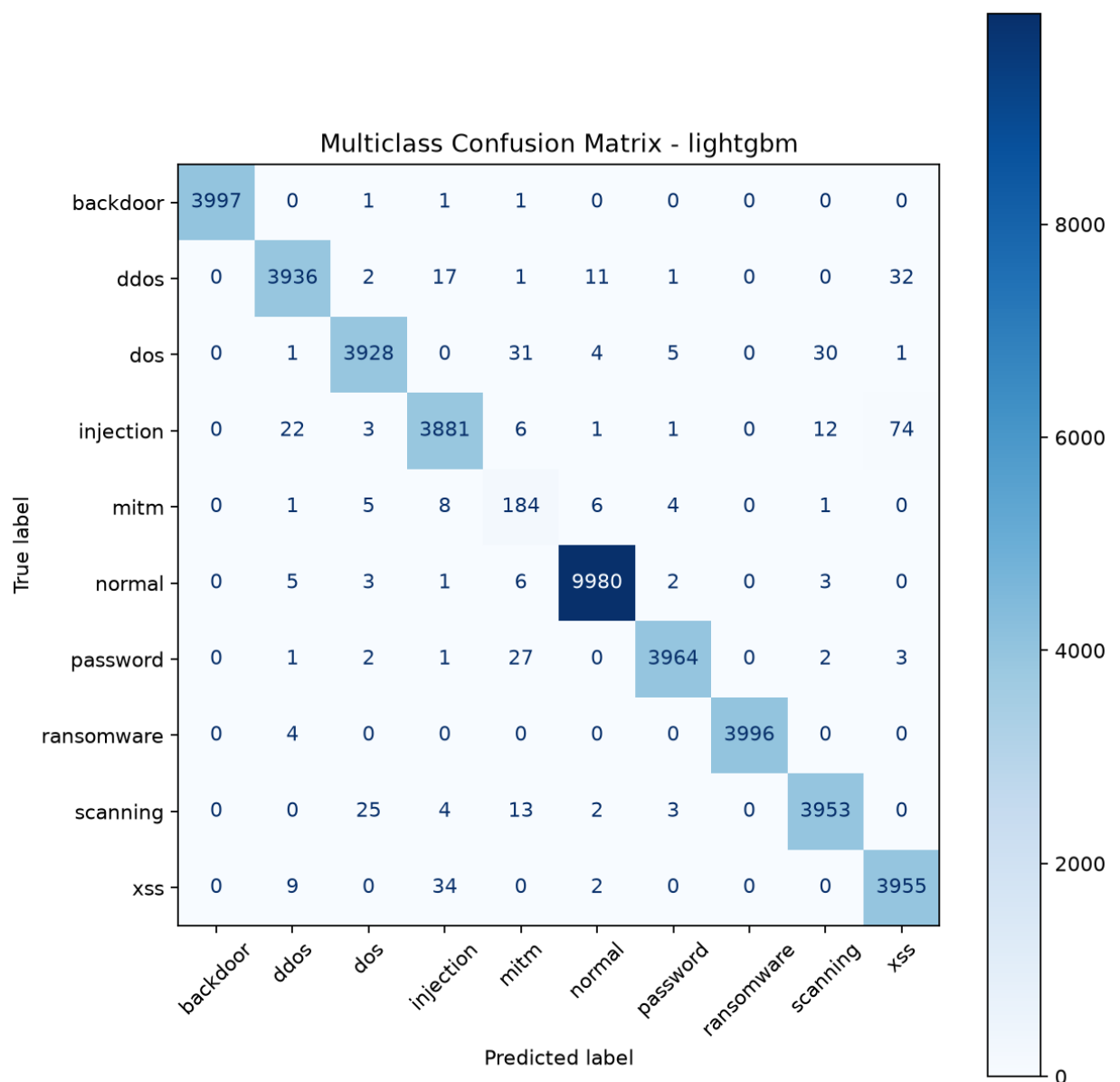


Figura 8 - Matrice di confusione LightGBM nella classificazione multiclasse

4.4 Effetto del bilanciamento tramite SMOTE

SMOTE è stato eseguito solo sul set di addestramento per evitare il data leakage. L'obiettivo è verificare se creare campioni sintetici per le classi meno rappresentate migliorerà (soprattutto per quanto riguarda il Macro F1). Ad esempio, nel caso binario Random Forest senza SMOTE è 0.9974 rispetto a con SMOTE 0.9969 Macro F1. Per la regressione logistica, il Macro F1 migliora da 0.9408 a 0.9410. Nel caso multiclass, LightGBM passa da 0.9524 senza SMOTE a 0.9528 con SMOTE. Il punto chiave è che SMOTE non ci porta un miglioramento sistematico. Per alcuni l'effetto è un piccolo positivo, per altri un piccolo negativo. Questo è un risultato interessante: il bilanciamento non dovrebbe essere una procedura predefinita, ma richiede tentativi ed

errori. Si addestra su un dataset bilanciato e, se le classi sono separabili linearmente o il modello riesce a gestire molto bene questo squilibrio, allora probabilmente non aiuterà in modo drastico.

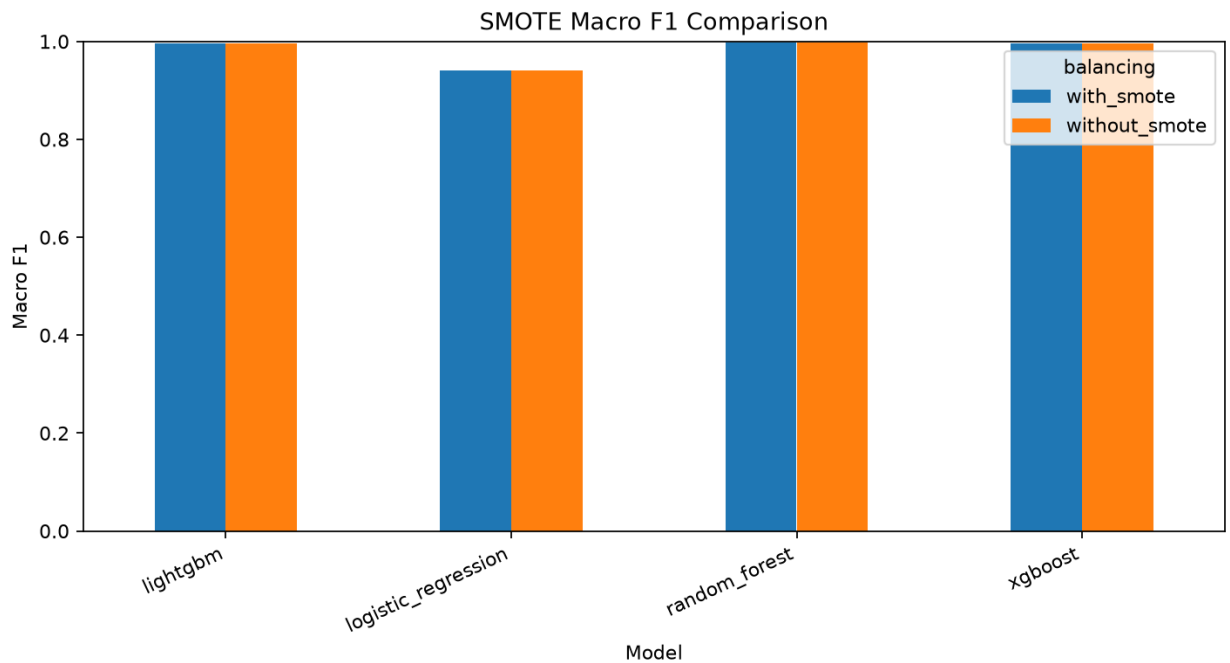


Figura 9 - Confronto Macro F1 con e senza SMOTE

4.5 Tuning iperparametrico

L'ottimizzazione degli iperparametri è stata eseguita su RandomizedSearchCV con 6 iterazioni e una validazione incrociata a 3 fold. È stato effettuato un campionamento stratificato del set di addestramento per limitare il costo computazionale. Questa scelta rende l'esperimento fattibile in un contesto locale, ma richiede cautela nell'interpretazione: mentre la baseline principale sfrutta l'intero set di addestramento, l'ottimizzazione opera su un sottoinsieme. Nel caso binario, LightGBM 0.9979 Macro F1; Random Forest 0.9974; XGBoost 0.9972. Per i task multiclass, Random Forest ottiene il punteggio migliore (Macro F1=0.9528). LightGBM si avvicina con 0.9523 e XGBoost ottiene un punteggio F1 di 0.9482. I valori sono alti, ma non si osserva un beneficio netto rispetto alla baseline complessiva. Questo risultato suggerisce due considerazioni. La prima è che le configurazioni della baseline sono già piuttosto competitive sul dataset considerato. La seconda è che un tuning esteso con più iterazioni, più fold o strumenti come Optuna potrebbe aiutare a proseguire il lavoro, ma richiederebbe tempi di calcolo molto maggiori.

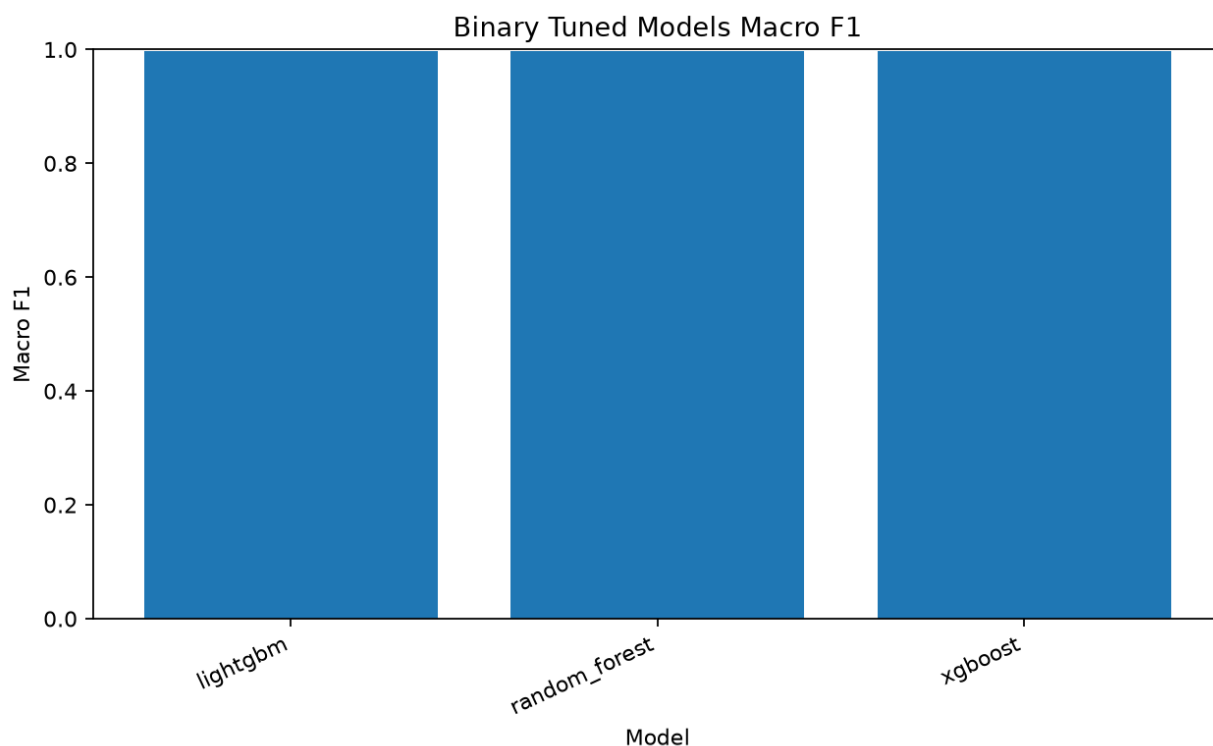


Figura 10 - Confronto Macro F1 dei modelli sottoposti a tuning binario

4.6 Modelli aggiuntivi: MLPClassifier e Isolation Forest

MLPClassifier è stato introdotto come modello neurale feed-forward. Nella classificazione binaria ottiene Macro F1 pari a 0.9899, mentre nella classificazione multiclasse ottiene Macro F1 pari a 0.8851. Il risultato binario è buono, ma rimane inferiore ai modelli ensemble migliori. Nel caso multiclasse, il modello presenta maggiori difficoltà e genera un warning di convergenza, indicando che l'ottimizzazione potrebbe richiedere più iterazioni, una diversa architettura o una migliore configurazione degli iperparametri.

È stata valutata una forma non supervisionata di rilevamento delle anomalie, ovvero Isolation Forest, per individuare anomalie binarie come mostrato. Il modello ottiene un valore di Macro F1 pari a 0,3625, molto inferiore rispetto a quello dei modelli supervisionati considerati. Questi risultati sono coerenti con la natura del metodo, poiché Isolation Forest è non supervisionato e quindi non sfrutta le etichette di attacco durante l'addestramento, disponendo pertanto di meno informazioni rispetto ai classificatori supervisionati. Tuttavia, la presenza dei risultati più bassi è una parte costante della discussione, non un errore sperimentale. Rivela che non tutti i modelli sono ugualmente validi nel risolvere il problema e che la natura supervisionata del dataset porta a una preferenza per modelli addestrati con etichette.

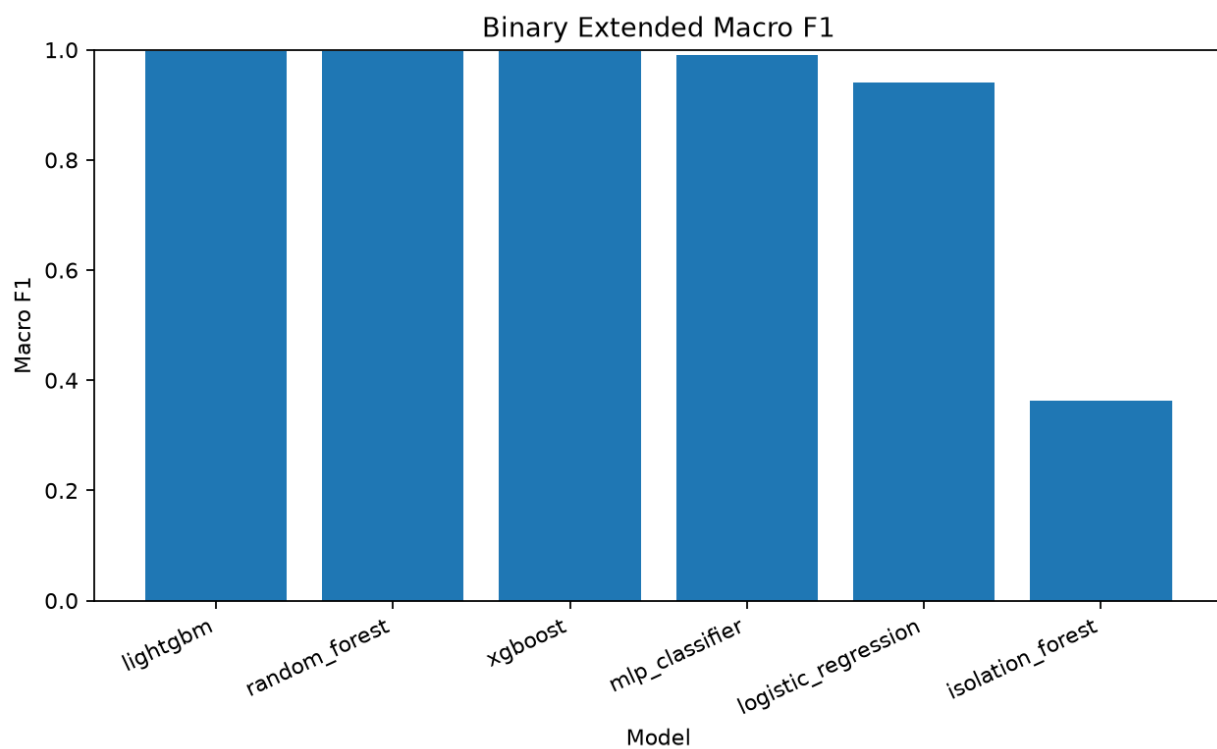


Figura 11 - Confronto Macro F1 degli esperimenti estesi binari

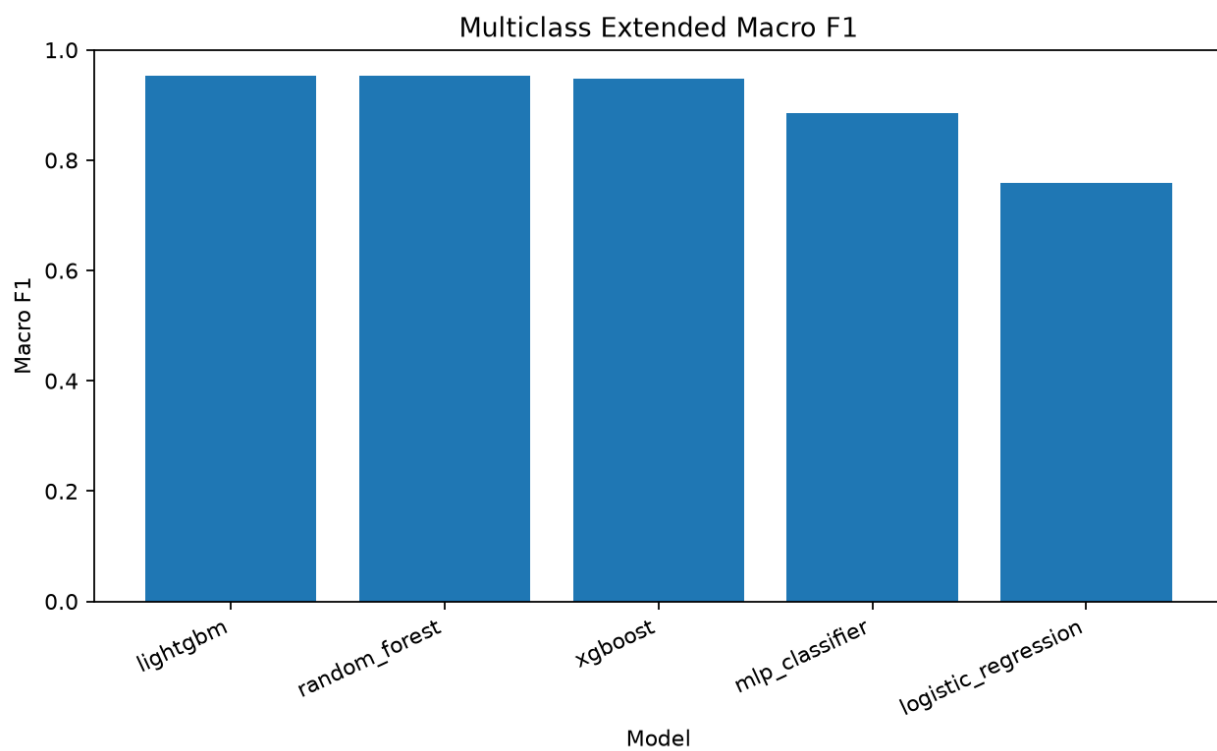


Figura 12 - Confronto Macro F1 degli esperimenti estesi multiclasse

4.7 Interpretabilità dei modelli

L'interpretabilità è un requisito importante per un IDS, soprattutto quando le decisioni del modello generano alert o interventi operativi. Un modello molto accurato ma completamente opaco può essere difficile da accettare in contesti reali, perché gli amministratori devono comprendere almeno quali feature contribuiscono maggiormente alle decisioni.

Nel progetto è stata eseguita un'analisi SHAP globale sul modello LightGBM, selezionato perché migliore baseline in base al Macro F1. L'importanza è calcolata come media del valore SHAP assoluto su 1.000 campioni del test set [10]. Nel caso binario, le feature principali sono `dst_port` (2.3037), `proto_tcp` (2.0220), `proto_udp` (1.8811), `conn_state_REJ` (1.4085) e `src_pkts` (1.2056). Nel caso multiclasse emergono `dst_port` (0.9983), `src_port` (0.8610), `src_ip_bytes` (0.6687), `duration` (0.6638) e `conn_state_REJ` (0.4237). Porte, protocolli, stato e volume del flusso sono caratteristiche coerenti con il dominio del traffico di rete.

L'analisi non deve essere interpretata come spiegazione causale completa: il valore SHAP quantifica quanto una feature contribuisce alle predizioni rispetto al valore di riferimento del modello, ma non dimostra da solo perché un evento sia malevolo. L'aggregazione assoluta adottata fornisce una lettura globale confrontabile tra feature; le spiegazioni locali di singoli alert rimangono fuori dal perimetro sperimentale dichiarato.

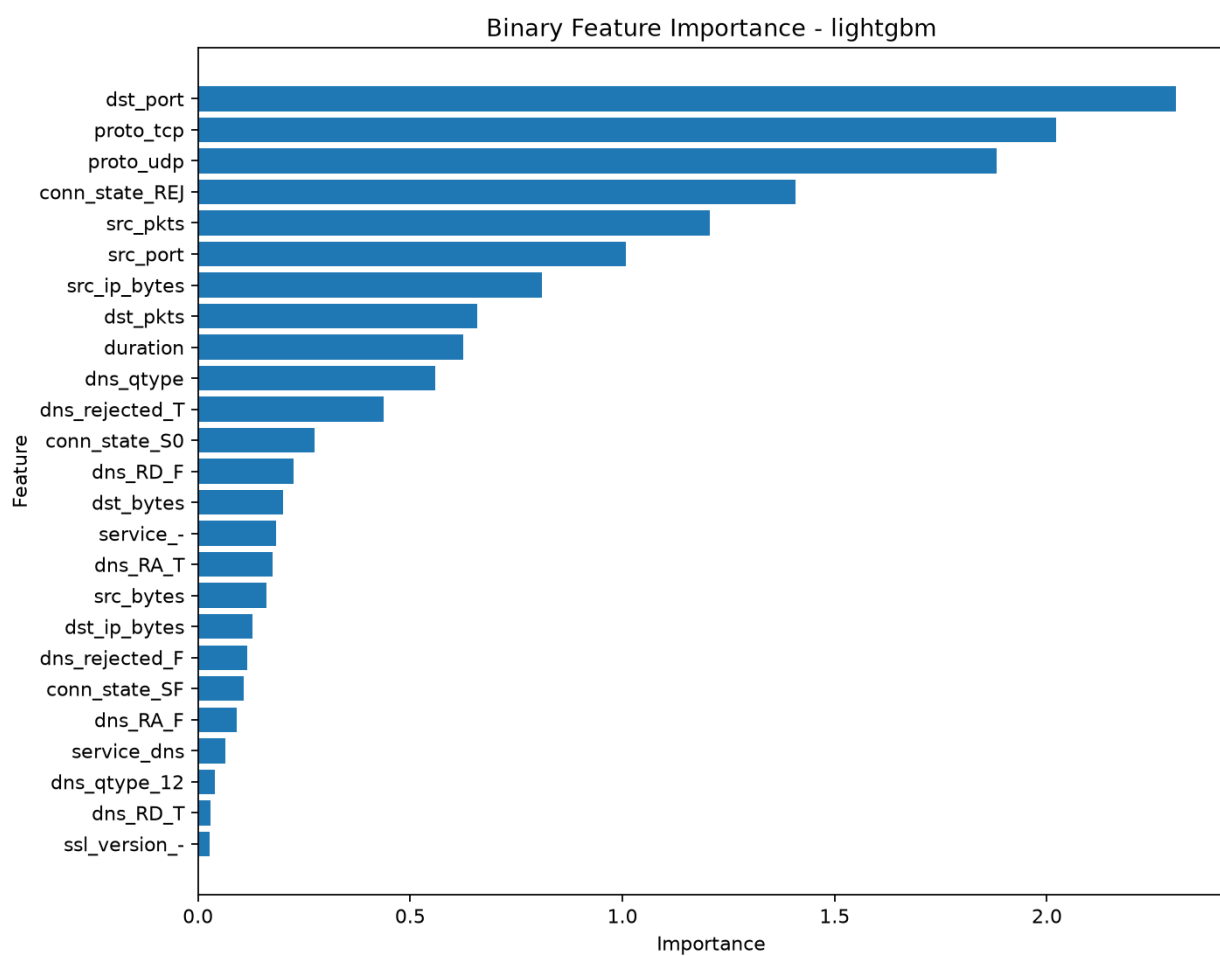


Figura 13 - Importanza delle feature nel modello LightGBM binario

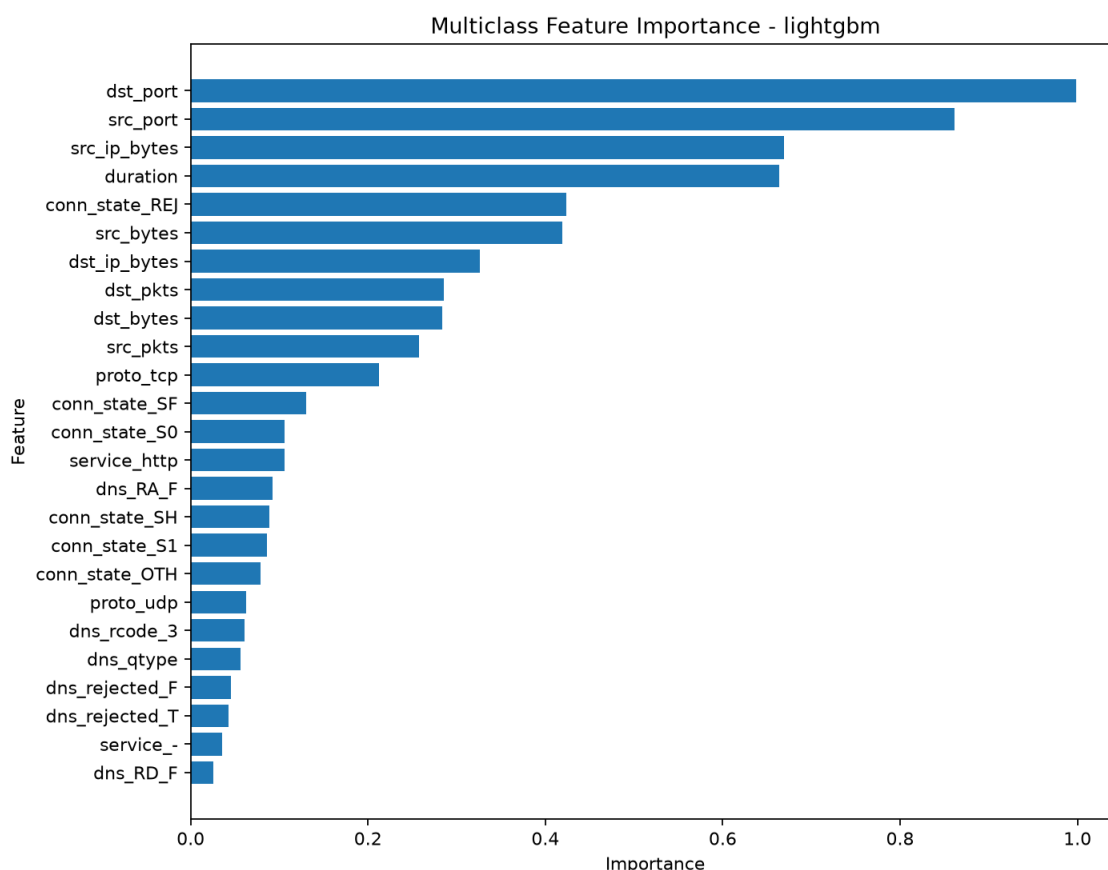


Figura 14 - Importanza delle feature nel modello LightGBM multiclasse

4.8 Latenza, dimensione e compressione

Oltre alle metriche predittive, sono state considerate anche misure relative al deployment. In un IDS pratico, il modello è tenuto a considerare il traffico in finestre temporali conformi a quanto è operativamente fattibile. Anche un modello accurato ma troppo lento o troppo grande potrebbe essere eccessivo per dispositivi edge o per un gateway a basso consumo. È stato utilizzato Joblib per l'analisi del deployment e si può vedere che gli artefatti di LightGBM riducono significativamente la dimensione; da 2,09 MB a 0,85 MB nel caso binario e da 17,20 MB a 7,34 MB nel caso multiclass. Questi valori riguardano la compressione dell'artefatto Python e non la quantizzazione per microcontrollori. Sono comunque utili per discutere il compromesso prestazioni/dimensione/portabilità.

La Regressione Logistica è la più veloce secondo le misurazioni di inferenza, ma ottiene anche meno previsioni corrette. Al contrario, LightGBM fornisce il miglior compromesso complessivo nella baseline, con buona accuratezza dei dati e tempi di

inferenza soddisfacenti. Tuttavia, in un caso pratico, la selezione del modello dipenderebbe dall'endpoint di deployment: cloud, gateway o dispositivo embedded.

4.9 Dimostratore embedded con Wokwi

È stato progettato un dimostratore Wokwi basato su ESP32 per emulare l'esecuzione embedded di un modello IDS di piccole dimensioni. Lo script di esportazione addestra una regressione logistica binaria su dieci feature numeriche, normalizza le variabili, quantizza coefficienti e intercetta in int16 e genera l'header C++ utilizzato dallo sketch Arduino.

L'exporter calcola quindi la differenza nell'accuratezza del test set tra la pipeline originale in floating-point di Python e una emulazione Python della formula quantizzata che verrebbe eseguita dal firmware su 42.209 record, per misurare l'impatto della quantizzazione. Entrambe le varianti hanno raggiunto accuratezza 0,8180, F1-score 0,8888 e Macro F1 0,6933. L'accordo sulla predizione è del 100% con una differenza di 0,0000 nell'accuratezza e nella Macro F1. Pertanto, al livello di quantizzazione adottato, l'arrotondamento non ha alcuna influenza sulle classi predette per questo set di test. Tuttavia, è importante notare che il confronto non tiene conto dell'effettivo consumo energetico o della latenza sull'hardware reale.

il progetto Wokwi è disponibile qui:

<https://wokwi.com/projects/472799587810026497>

Il dimostratore embedded è costituito da tre file principali: sketch.ino, embedded_model.h e diagram.json. Lo sketch imposta una connessione seriale, carica i campioni di test, calcola la probabilità di attacco, ne stampa l'etichetta attesa e l'etichetta predetta per ciascun campione insieme alle rispettive probabilità. Il file diagram.json collega i pin seriali dell'ESP32 al monitor seriale virtuale, consentendo di controllare l'output durante la simulazione.

Il dimostratore non usa LightGBM. Il motivo di questa scelta è voluto: LightGBM è il modello più potente del baseline, ma ha una complessità maggiore che renderebbe più difficile il deployment così com'è in un puro sketch per microcontrollore. A questo proposito, si potrebbe rappresentare l'inferenza in un modello lineare, come un prodotto scalare di feature normalizzate e coefficienti quantizzati, seguito dalla funzione sigmoide, piuttosto che trattarla come somma di valori attesi pesati. L'output è meno

preciso, ma è meglio adatto a dimostrare il diverso percorso concettuale tra una pipeline Python e l'inferenza embedded.

Possiamo vedere nell'output seriale del simulatore che effettua comunque previsioni corrette e presenta errori. La significatività di questo è che mostra che la sequenza altrimenti molto artificiale non rappresenta un adattamento perfetto del dimostratore, come ci si aspetterebbe più strettamente che la performance di un ensemble (baseline) sia superiormente più alta rispetto a quando abbiamo usato un modello più compresso o con prestazioni inferiori per le inferenze finali a partire dall'input.

4.10 Sintesi degli artefatti prodotti

Gli esperimenti hanno prodotto file CSV, grafici PNG e documentazione tecnica. I principali file di metriche sono `binary_baseline_metrics.csv`, `multiclass_baseline_metrics.csv`, `binary_extended_metrics.csv`, `multiclass_extended_metrics.csv`, `smote_comparison.csv`, `hyperparameter_tuning_results.csv`, `latency_summary.csv`, `deployment_analysis.csv` ed `embedded_logistic_regression_metrics.csv`.

I grafici principali includono confronti per accuracy, F1-score, Macro F1, matrici di confusione, distribuzioni delle classi e feature importance. Tali figure sono incluse nel documento per rendere immediata la lettura dei risultati e per collegare le metriche numeriche alla loro rappresentazione visuale.

La presenza di artefatti generati automaticamente rafforza la riproducibilità del lavoro. Un lettore può confrontare le tabelle riportate nel testo con i CSV nella directory `results/metrics/` e verificare che le immagini inserite provengano dai file presenti in `results/plots/`.

4.11 Lettura critica dei risultati

Anche i modelli a ensemble devono essere interpretati con attenzione, poiché l'elevatissima prestazione indica, da un lato, che le caratteristiche del dataset sono informative e che i modelli potrebbero discriminare bene tra traffico normale e malevolo. D'altra parte, un valore molto alto per dataset statici potrebbe essere determinato dalla struttura del dataset, dalla separabilità delle classi e per finire dalla loro somiglianza tra i set di addestramento e di test. È esattamente per questo che il lavoro non afferma che un modello addestrato su TON_IoT possa essere utilizzato così

com'è per difendere reti live. Mostra che la pipeline è effettivamente valida e che i modelli apprendono dal dataset considerato attraverso un risultato sperimentale. Per essere convalidato in ambienti reali, sarebbe necessario raccogliere e testare ulteriori dati lungo l'intero ciclo di vita delle implementazioni su traffico reale e su concetti al di fuori dello spazio delle classi presenti nel training.

Un altro componente chiave è la prestazione predittiva rispetto all'operatività. Un IDS ad alta accuratezza può comunque presentare importanti falsi positivi o falsi negativi.

Capitolo 5 - Discussione e strategie di mitigazione

5.1 Interpretazione complessiva dei risultati

I risultati mostrano che i modelli di ensemble hanno successo in modo specifico sul dataset TON_IoT. LightGBM: accuratezza più alta, punteggio F1 e Macro F1 sia nelle basi binarie sia in quelle multiclasse. Random Forest e XGBoost restano comunque validi concorrenti; la regressione logistica mostra invece limiti più evidenti, soprattutto nel caso multiclasse. Il risultato di LightGBM può essere spiegato dal suo approccio: modella solo le relazioni non lineari tra le feature tabellari e le classi. Il traffico di rete contiene relazioni complesse tra porte, durata, pacchetti e byte; i modelli basati su alberi possono catturare tali relazioni più facilmente di un modello lineare.

La distinzione tra classificazione binaria e multiclasse è fondamentale. Nel caso binario, il modello deve classificare un campione come benigno o maligno. Nel caso multiclasse, deve differenziare tra alcune classi, alcune delle quali possono essere simili per natura. È per questo che il problema multiclasse è più difficile e il Macro F1 diventa ancora più significativo.

5.2 Interpretabilità contro prestazioni

Un IDS deve essere preciso, ma anche comprensibile. Un allarme in un ambiente operativo deve essere interpretabile, azionabile e gestibile. Sebbene possano ottenere prestazioni elevate, i modelli di ensemble sono meno interpretabili della regressione logistica. Un'analisi SHAP attenua questo problema in una certa misura, identificando tramite un criterio coerente quali variabili esercitano la maggiore influenza sulle predizioni aggregate [10]. Poi la regressione logistica è meno accurata, ma i meccanismi sono molto più semplici. I coefficienti del modello possono essere interpretati direttamente e l'uso per l'inferenza è computazionalmente poco costoso. Questo la rende interessante per casi d'uso embedded o didattici, come quello dimostrato con questo esempio Wokwi. Tuttavia, nel dataset preso in considerazione, imporre tale semplicità ha un costo: il modello lineare non raggiunge le prestazioni degli ensemble.

Quindi la scelta del modello dovrebbe essere contestuale. Su server o gateway dotati di risorse adeguate, LightGBM può rappresentare la scelta più efficace; sui dispositivi più

limitati occorre invece accettare un compromesso prestazionale o ricorrere a tecniche TinyML [19].

5.3 Limiti del dataset statico

Il dataset TON_IoT è adeguato al confronto controllato dei modelli, ma rimane un insieme di dati statico, raccolto in un contesto temporale definito e non aggiornato continuamente. Nelle reti reali il traffico cambia nel tempo. Vengono aggiunti nuovi dispositivi, viene aggiornato il firmware, cambiano le configurazioni, cambiano i comportamenti degli utenti e avvengono nuovi attacchi. Questo fenomeno, chiamato concept drift, può portare a una riduzione delle prestazioni di un modello addestrato su dati passati.

Un ulteriore limite riguarda la rappresentatività. Quando un modello è addestrato su alcuni dati, potrebbe imparare caratteristiche specifiche di quel dataset e fallire nel generalizzare bene in altri ambienti. Ecco perché una valutazione più approfondita dovrebbe includere diversi dataset, traffico raccolto in laboratorio e test in scenari realistici. Quindi la pipeline che è stata sviluppata riguarda baseline sperimentali, non soluzioni. Il contributo chiave del lavoro risiede nella sua riproducibilità; gli esperimenti possono essere riprodotti, adattati e integrati.

5.4 Limiti della simulazione Wokwi

Il dimostratore Wokwi è valido per mostrare un possibile primo passo verso l'inferenza embedded, con alcune limitazioni. La simulazione non consente misurazioni, senza alcun impatto sul consumo energetico reale, sulla temperatura, sulle interferenze, sulla latenza di rete fisica o sui vincoli completi dell'hardware. Inoltre, si utilizzano campioni statici esportati per il modello embedded e non traffico acquisito in tempo reale da un'interfaccia di rete. Nonostante questi ostacoli, Wokwi è prezioso per tutta la tesi perché consente di dimostrare come il codice embedded funzioni senza hardware reale. In futuro si potrebbe eseguire l'esperimento su dispositivi reali, con misurazioni della memoria occupata, del tempo di inferenza, della stabilità e del consumo energetico.

5.5 Strategie di mitigazione

Un IDS basato su machine learning deve essere combinato con una strategia di difesa olistica. Non può sostituire la prevenzione, ma può aiutare fornendo rilevamento. E la segmentazione di rete, il principio del minor privilegio, gli aggiornamenti del firmware, l'autenticazione forte, l'inventario dei dispositivi e il monitoraggio del traffico con

rilevamento delle anomalie, registrazione e gestione degli avvisi e zero trust sono le principali strategie di mitigazione [3], [17], [18]. La segmentazione di rete inibisce il movimento laterale.

Un'ulteriore strategia sarebbe non mettere mai i dispositivi IoT sulla stessa VLAN o rete di sistemi critici, server e workstation degli utenti. Il principio del privilegio minimo limita i permessi dei dispositivi, degli account e dei servizi. Password robuste e coppie di chiavi crittografiche riducono il rischio di accessi non autorizzati, in particolare ai broker MQTT, alle dashboard e alle API. Gli aggiornamenti del firmware e le correzioni delle vulnerabilità note devono essere gestiti con attenzione, soprattutto in ambienti industriali in cui un'interruzione del servizio può essere costosa. L'inventario dei dispositivi consente di conoscere la versione del firmware e se il dispositivo è cablato o wireless. Una gestione efficace della sicurezza non può essere sfruttata senza l'inventario. Anche il monitoraggio del traffico e il logging centralizzato per il rilevamento delle anomalie e la ricostruzione degli incidenti sono utili. Infine, un IDS può generare avvisi, che devono essere integrati in un processo che includa classificazione della gravità, verifica della risposta, escalation e un ciclo di feedback di regole o modelli.

5.6 Possibile percorso verso una validazione su hardware reale

Il passaggio da simulazione a hardware reale richiede una sequenza di attività progressive. In primo luogo, occorre scegliere il dispositivo target, ad esempio un ESP32 o un gateway edge più potente. In secondo luogo, occorre stabilire quali feature siano effettivamente disponibili sul dispositivo. Un modello addestrato su feature di flusso di rete non può essere eseguito direttamente su un sensore se quel sensore non osserva tali feature.

In terzo luogo, è necessario definire come raccogliere i dati in tempo reale. Un dispositivo embedded potrebbe ricevere feature già calcolate da un gateway, oppure potrebbe calcolare solo indicatori semplici. In quarto luogo, occorre misurare latenza, memoria e consumo energetico. Queste misure sono fondamentali perché un modello accurato ma troppo costoso potrebbe non essere adatto al deployment.

Il dimostratore Wokwi copre solo una parte di questo percorso: mostra che una forma compatta del modello può essere eseguita in un ambiente simulato. La validazione su

dispositivi reali richiederebbe esperimenti aggiuntivi in laboratorio, con misurazioni fisiche e traffico generato o raccolto in condizioni controllate.

5.7 Indicazioni operative per un IDS IoT

I principali aspetti da considerare per migliorare le prestazioni degli IDS per IoT non sono complesse. Tutto parte dall'inventario dei dispositivi: non è possibile definire il comportamento atteso e le priorità senza sapere quali dispositivi sono presenti. La segmentazione nell'IoT dovrebbe interagire solo con quei servizi che sono effettivamente necessari. Come terzo punto è necessario raccogliere log e flussi di rete. Dopo la raccolta dei dati, la generazione degli avvisi deriva dal modello dell'IDS. Tuttavia, gli avvisi devono essere prioritizzati e mappati su flussi di risposta. I falsi positivi possono essere gestiti sia tramite revisione manuale sia regolando le soglie di rilevamento; i falsi negativi richiedono un'analisi delle cause profonde, che a sua volta richiede l'aggiornamento del modello o delle regole. Ecco perché la sicurezza non è un singolo evento, ma un'attività continua. Richiede inoltre una procedura per gli aggiornamenti. Se un modello viene addestrato una sola volta, diventerà obsoleto. I nuovi dispositivi e i nuovi attacchi richiedono la raccolta di nuovi dati, la rivalutazione delle metriche e la distribuzione di nuove versioni del modello in modo controllato.

5.8 Considerazioni etiche e di gestione dei dati

I dati IoT possono contenere informazioni sensibili e anche il monitoraggio del traffico oggetto di monitoraggio. Anche se il set di dati utilizzato nella tesi è pubblico e scaricato per scopi di ricerca, da un punto di vista reale la raccolta dovrebbe seguire i principi di minimizzazione, protezione e controllo degli accessi. I dati acquisiti da dispositivi domestici, sanitari o industriali possono mostrare abitudini, modalità di lavoro e informazioni sensibili. I dati di log devono essere limitati ai log pertinenti, i log devono essere protetti e le politiche di conservazione in atto dovrebbero limitare l'accesso ai log solo al personale autorizzato. In effetti, la sicurezza del sistema di rilevamento fa parte anche del problema: un IDS compromesso potrebbe divulgare informazioni preziose o persino servire a nascondere attacchi.

Capitolo 6 - Conclusioni

La tesi ha affrontato il problema di sicurezza nei sistemi IoT con un percorso sperimentale basato sui Sistemi di Rilevamento delle Intrusioni e sul ML. Il lavoro ha portato a un repository riproducibile, strutturato e documentato che può essere utilizzato per esperimenti di classificazione binaria e multiclasse sul dataset TON_IoT. Come panoramica di base è stato confrontato Random Forest, LightGBM, XGBoost e Logistic Regression. Questi risultati indicano che i modelli di ensemble, come LightGBM in questo caso di studio, ottengono prestazioni estremamente elevate.

Risultati: non i più accurati, ma la regressione logistica è un modello semplice e interpretabile, adatto per un compito dimostrativo embedded. Il confronto è stato ampliato e sono stati discussi limiti e vantaggi dei diversi approcci con esperimenti estesi che includono SMOTE, tuning, MLPClassifier e Isolation Forest. L'analisi SHAP e le misure di deployment hanno spinto la valutazione oltre le metriche predittive. Il dimostratore Wokwi è stato dimostrato che l'export e la simulazione di un modello compatto su ESP32 sono possibili, ma anche per mostrare il compromesso tra accuratezza e portabilità.

Questo non dovrebbe essere visto come una risposta finale in relazione alla sicurezza IoT, ma piuttosto come un elemento di un esperimento condotto entro uno specifico ambito definito. Estensioni autonome che non dipendono dal completamento del lavoro di tesi includono test su dataset aggiuntivi, analisi del concept drift, raccolta di traffico reale e riproduzione di hardware fisico.

Questo ultimo passo potrebbe consentire di mappare aspetti che non sono misurabili in simulazione: memoria reale, latenze effettive, consumo energetico e stabilità del dispositivo.

Infine, l'intera tesi fornisce evidenze che i modelli di machine learning possono agire come strumenti efficaci per il rilevamento delle intrusioni negli ambienti IoT quando vengono inseriti in una metodologia di sicurezza più ampia e valutati a fondo rispetto a riproducibilità, interpretabilità, limiti del dataset e vincoli di deployment.

Bibliografia

- [1] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood e A. Anwar, "TON_IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven intrusion detection systems", *IEEE Access*, vol. 8, pp. 165130-165150, 2020, doi: 10.1109/ACCESS.2020.3022862.
- [2] UNSW Canberra, "The TON_IoT Datasets", <https://research.unsw.edu.au/projects/toniot-datasets>.
- [3] ENISA, "Baseline Security Recommendations for IoT in the context of Critical Information Infrastructures", 2017.
- [4] OASIS, "MQTT Version 5.0", OASIS Standard, 2019.
- [5] L. Breiman, "Random Forests", *Machine Learning*, vol. 45, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.
- [6] T. Chen e C. Guestrin, "XGBoost: A Scalable Tree Boosting System", *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, doi: 10.1145/2939672.2939785.
- [7] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", *Advances in Neural Information Processing Systems* 30, 2017.
- [8] N. V. Chawla, K. W. Bowyer, L. O. Hall e W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique", *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002, doi: 10.1613/jair.953.
- [9] F. T. Liu, K. M. Ting e Z.-H. Zhou, "Isolation Forest", 2008 Eighth IEEE International Conference on Data Mining, 2008, doi: 10.1109/ICDM.2008.17.
- [10] S. M. Lundberg e S.-I. Lee, "A Unified Approach to Interpreting Model Predictions", *Advances in Neural Information Processing Systems* 30, 2017.
- [11] Docker documentation, "Docker Compose", <https://docs.docker.com/compose>.
- [12] Wokwi Docs, "diagram.json File Format" e "Project Configuration", <https://docs.wokwi.com/>.
- [13] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets", *Sustainable Cities and Society*, vol. 72, art. 102994, 2021, doi: 10.1016/j.scs.2021.102994.
- [14] T. M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa e F. T. H. den Hartog, "ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets", *IEEE Internet of Things Journal*, vol. 9, n. 1, pp. 485-496, 2022, doi: 10.1109/JIOT.2021.3085194.
- [15] S. Sicari, A. Rizzardi, L. A. Grieco e A. Coen-Porisini, "Security, privacy and trust in Internet of Things: The road ahead", *Computer Networks*, vol. 76, pp. 146-164, 2015, doi: 10.1016/j.comnet.2014.11.008.
- [16] B. B. Zarpelão, R. S. Miani, C. T. Kawakani e S. C. de Alvarenga, "A survey of intrusion detection in Internet of Things", *Journal of Network and Computer Applications*, vol. 84, pp. 25-37, 2017, doi: 10.1016/j.jnca.2017.02.009.
- [17] NIST, "Foundational Cybersecurity Activities for IoT Device Manufacturers", NISTIR 8259, 2020, doi: 10.6028/NIST.IR.8259.
- [18] S. Rose, O. Borchert, S. Mitchell e S. Connelly, "Zero Trust Architecture", NIST Special Publication 800-207, 2020, doi: 10.6028/NIST.SP.800-207.
- [19] P. Warden e D. Situnayake, "TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers", O'Reilly Media, 2019.

- [20] Scikit-learn Developers, "LogisticRegression", scikit-learn API Reference, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.
- [21] Scikit-learn Developers, "MLPClassifier", scikit-learn API Reference, https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html.